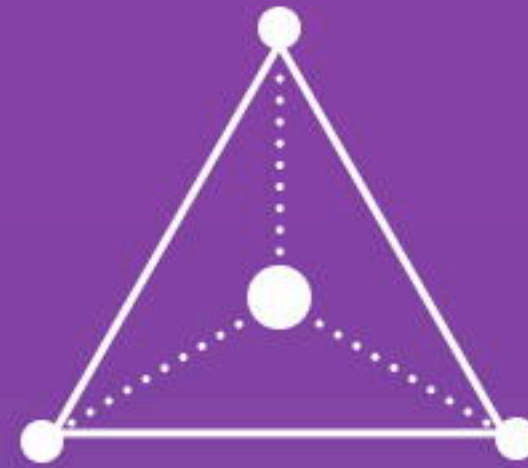


PORTFOLIO

Carlos López, Game Designer



carjelosa@gmail.com



linkedin.com/in/carjelosa/

Genera Games

Game Designer
Level Designer
FX Artist
Game Developer

Overview

Genera Games is a game company focused in delivering F2P games for Android and iOS markets all around the world.

Many of the games delivered had to be **upgraded** since their game monetization mechanics were outdated; new content had to be introduced in order to increase their **retention** and **visibility** in the app stores.

My mission was to take 4 games and upgrade them: **design** new game loops, **create** new content and **implement** them to test their feasibility.



The Little Prince



Runbot



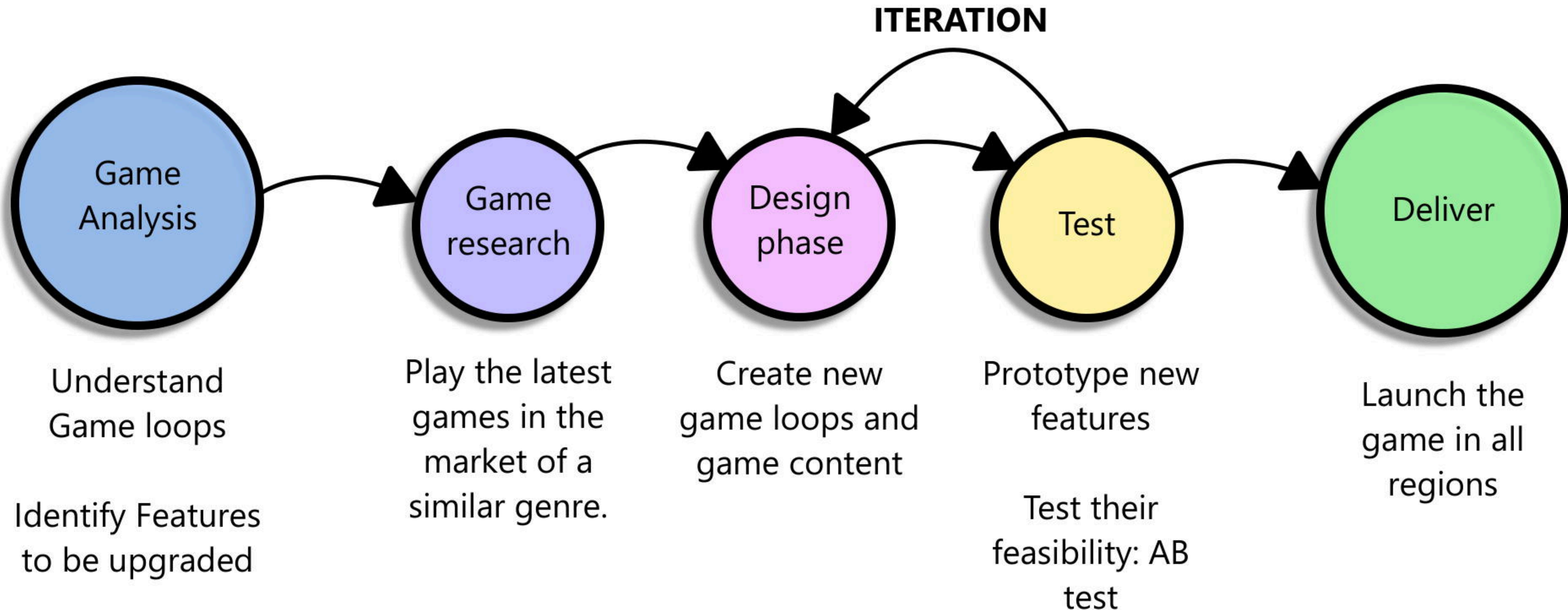
Xtreme Racing 2



SkullTowers

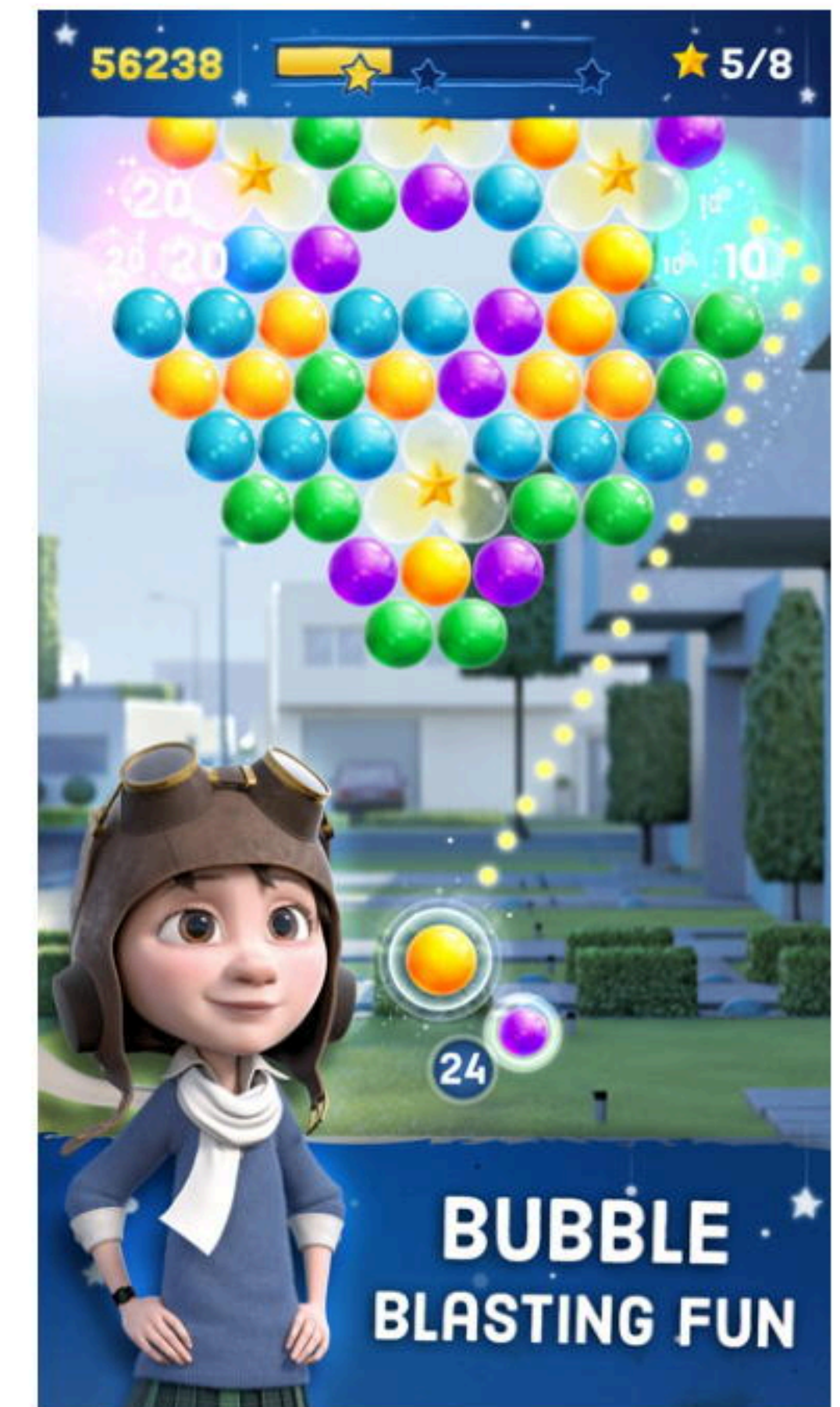
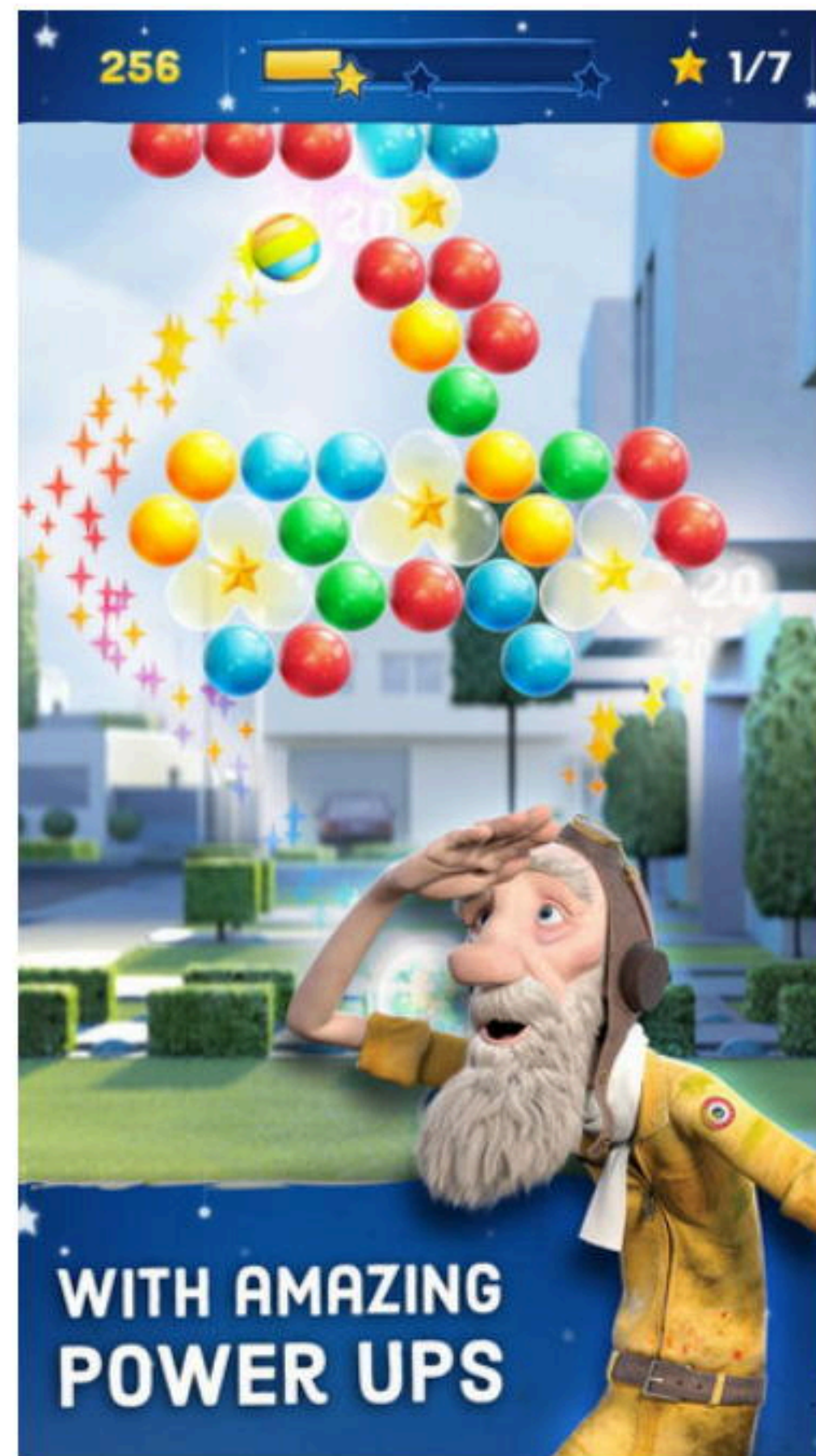
Process

Although each one of these games had its own challenges, the upgrade process for each one of them was the same.

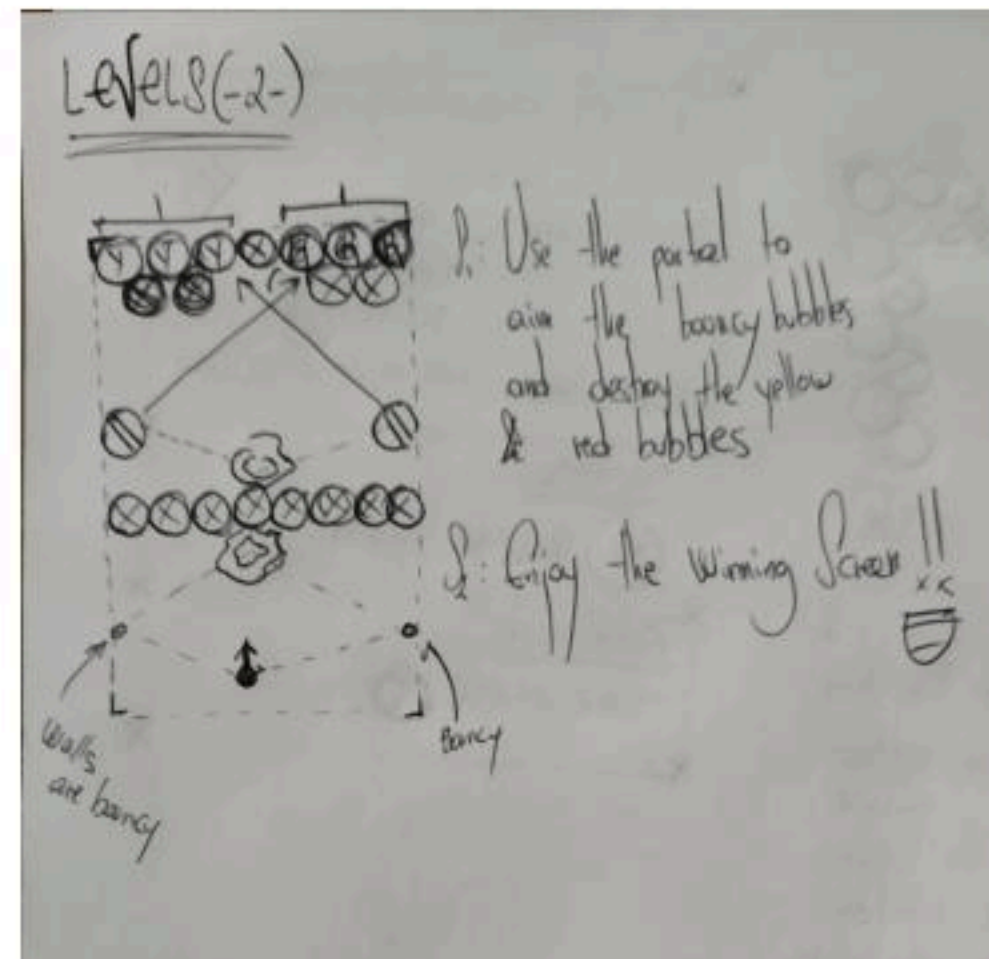
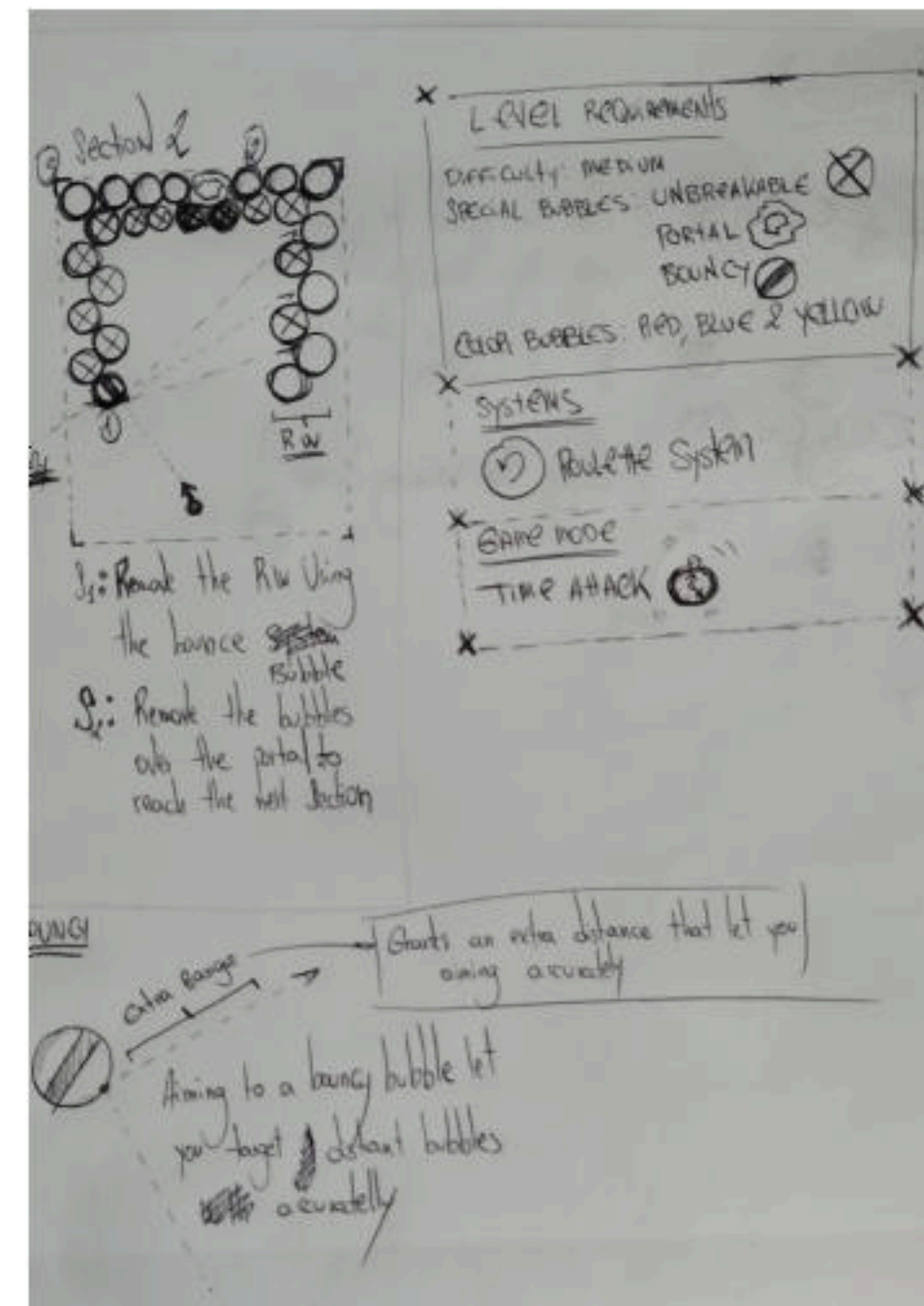
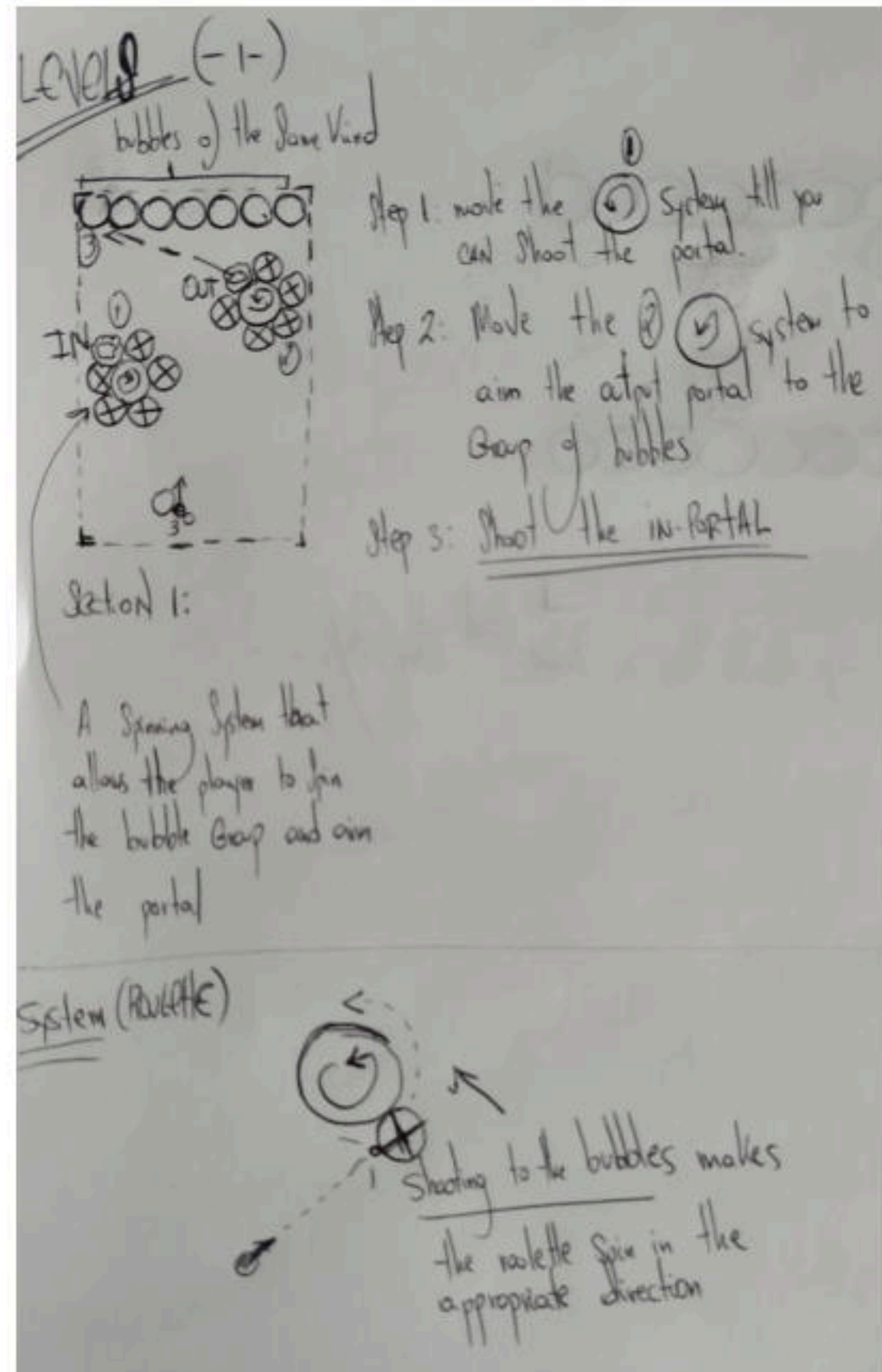


The Little Prince -Bubble Pop-

A bubble pop game with different game modes: time attack, puzzle mode, classic mode and many others.
For this project, my main tasks were: designing new levels and new game modes for the second part of this game.



Game Levels



The Little Prince -Bubble Pop-

For each level, there was a set of requirements to be followed in the level design process:

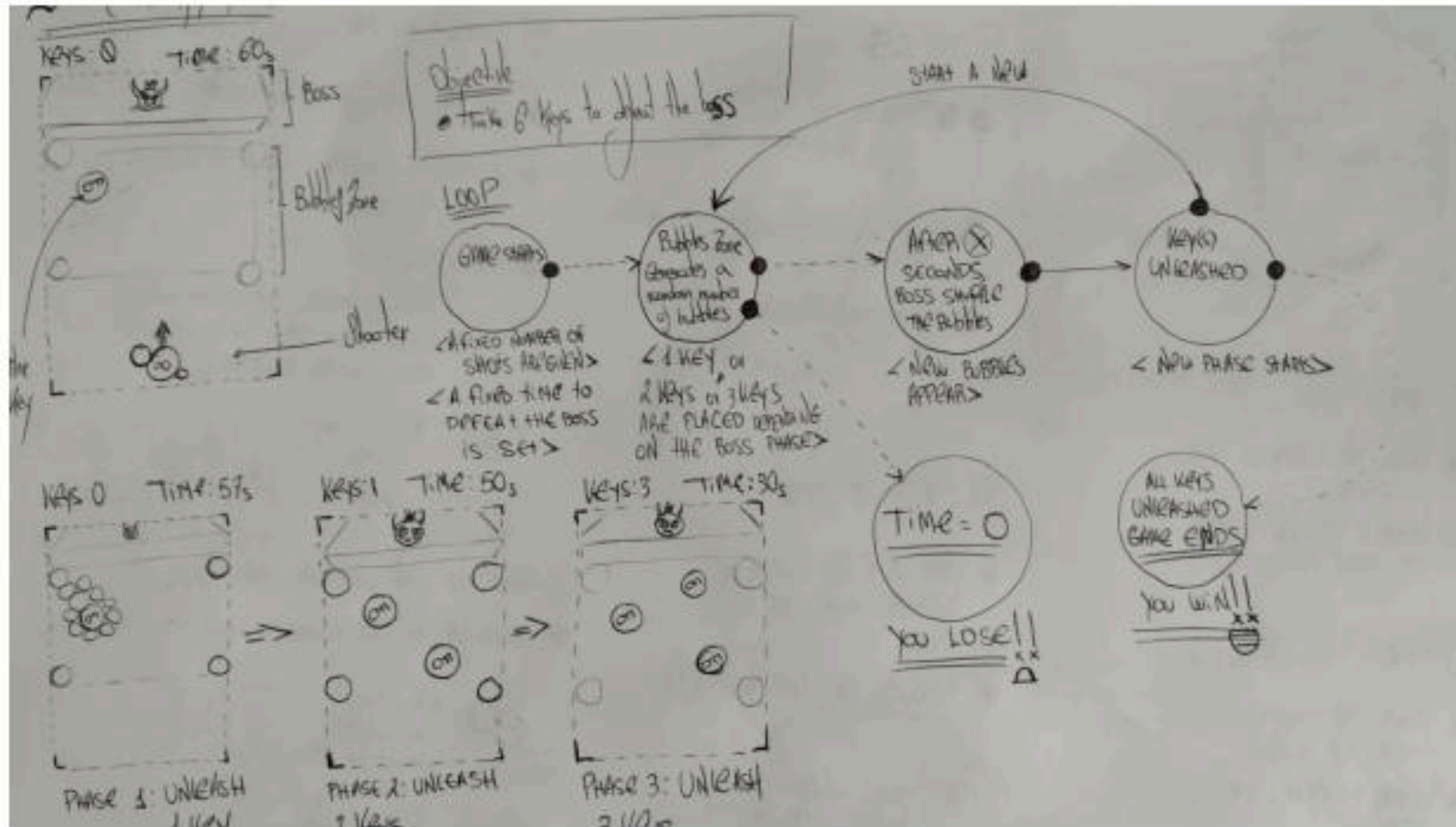
- Level Difficulty
- Special bubbles to be implemented: Unbreakable, Portals, Bouncy, Red, Green, Yellow etc...
- Color bubbles to be implemented: Yellow, Green, Purple etc...
- Game Mode: Classic mode, Star mode, Time Attack...

My process started on papers, making different sketches for each section of the level, thinking about how it could be solved and if the different gameplay systems could fit each other.

After this process, the level was prototyped directly inside Unity Engine. Taking these sketches into consideration, several levels were prototyped and played with the lead game designer to test the game feel and functionalities. The most suitable one was selected to be the final one.

This process was the same for each level of the game. In this way we could benefit for the discarded levels if we wanted to introduce new content for the game and even improve it.

Game Modes



The Little Prince -Bubble Pop-

For the next episode of this game, we had to design new and different game modes to be introduced.

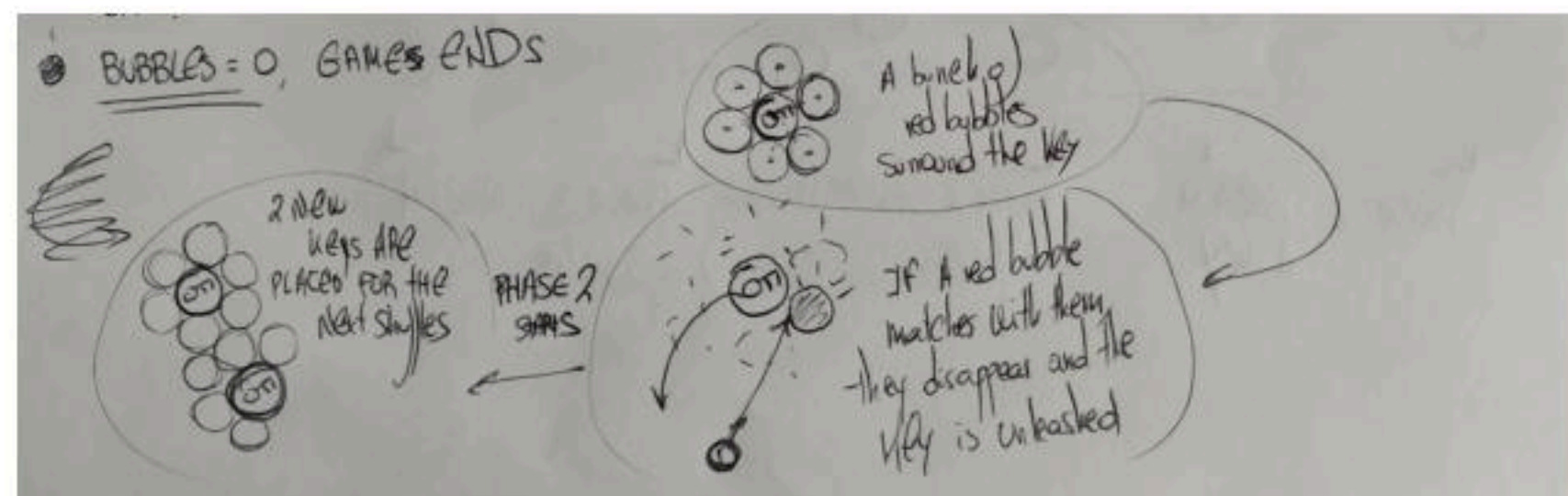
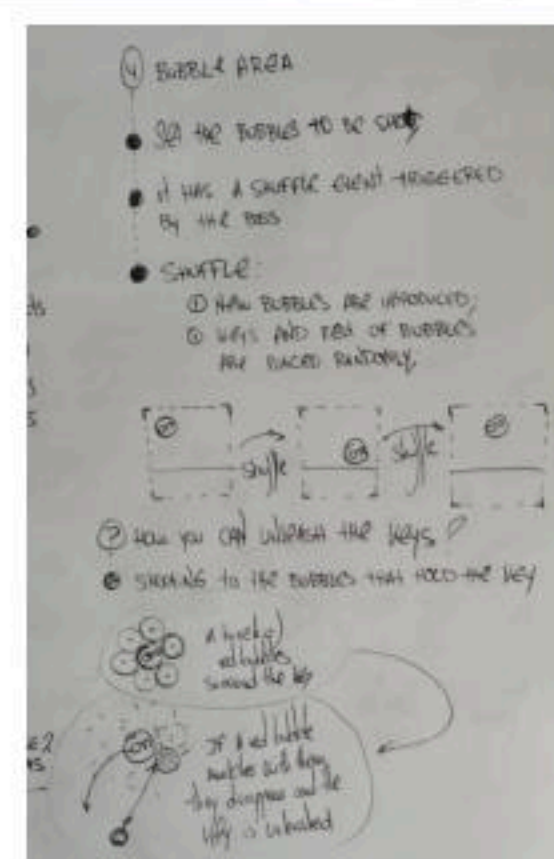
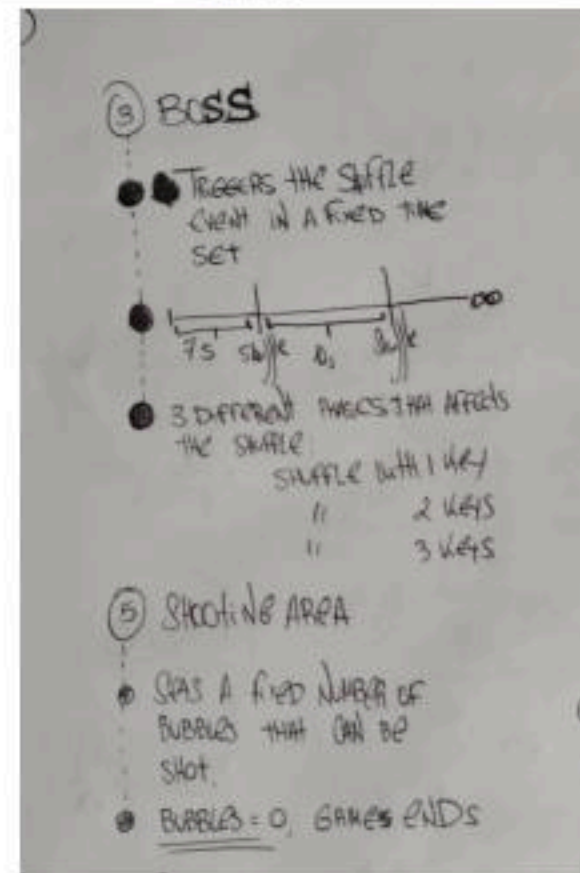
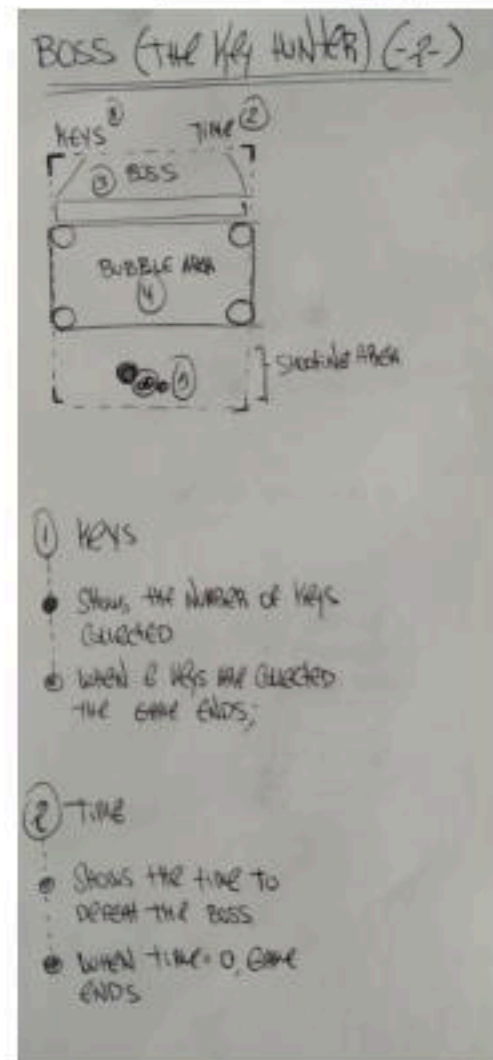
I had the freedom to design one for the bubble boss fight.

I started sketching and prototyping on paper the global behavior of the game mode.

Then a deep analysis of each game system was made to see if all of them could match properly with each other and maintain an holistic feeling through the gameplay.

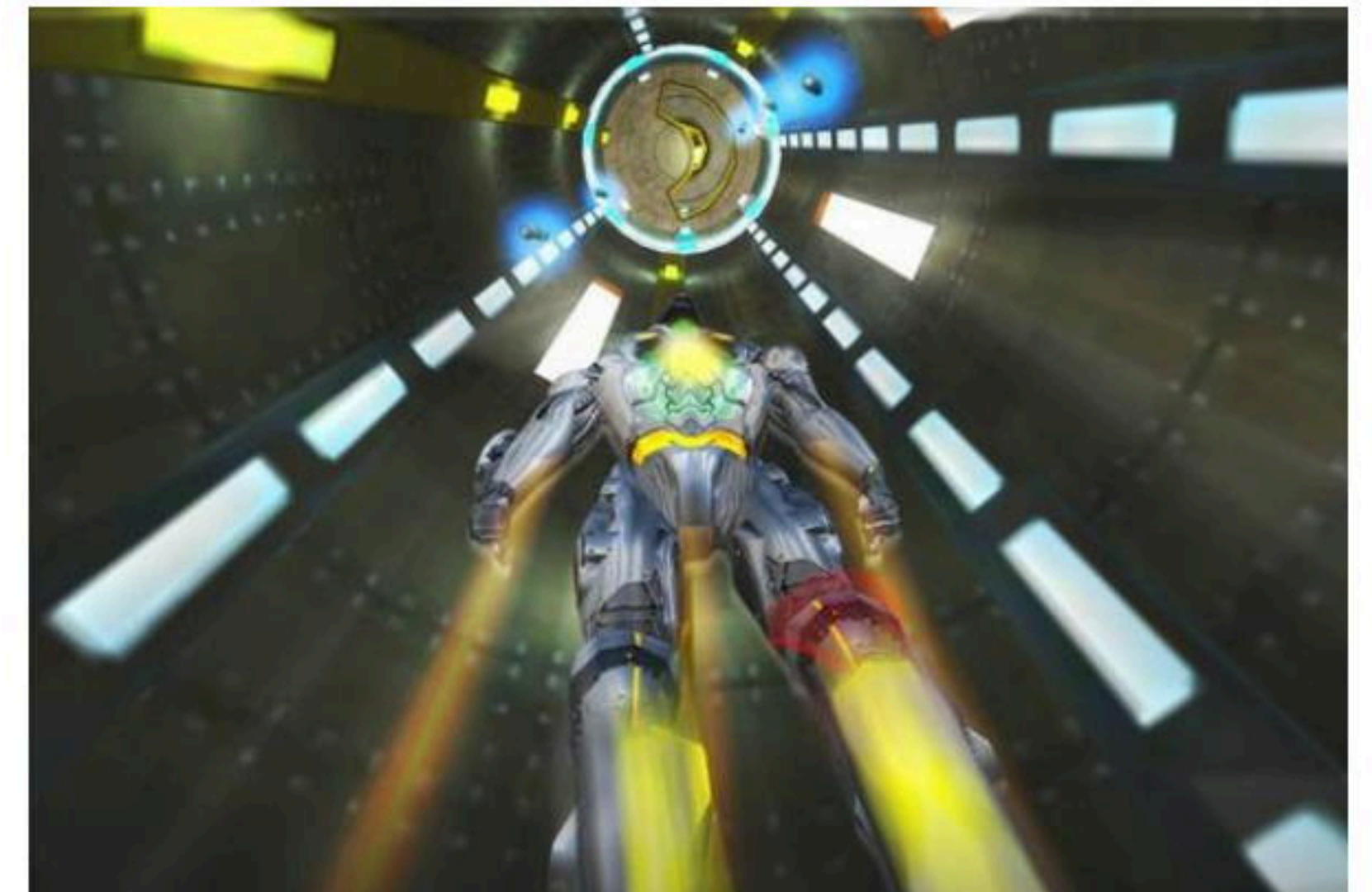
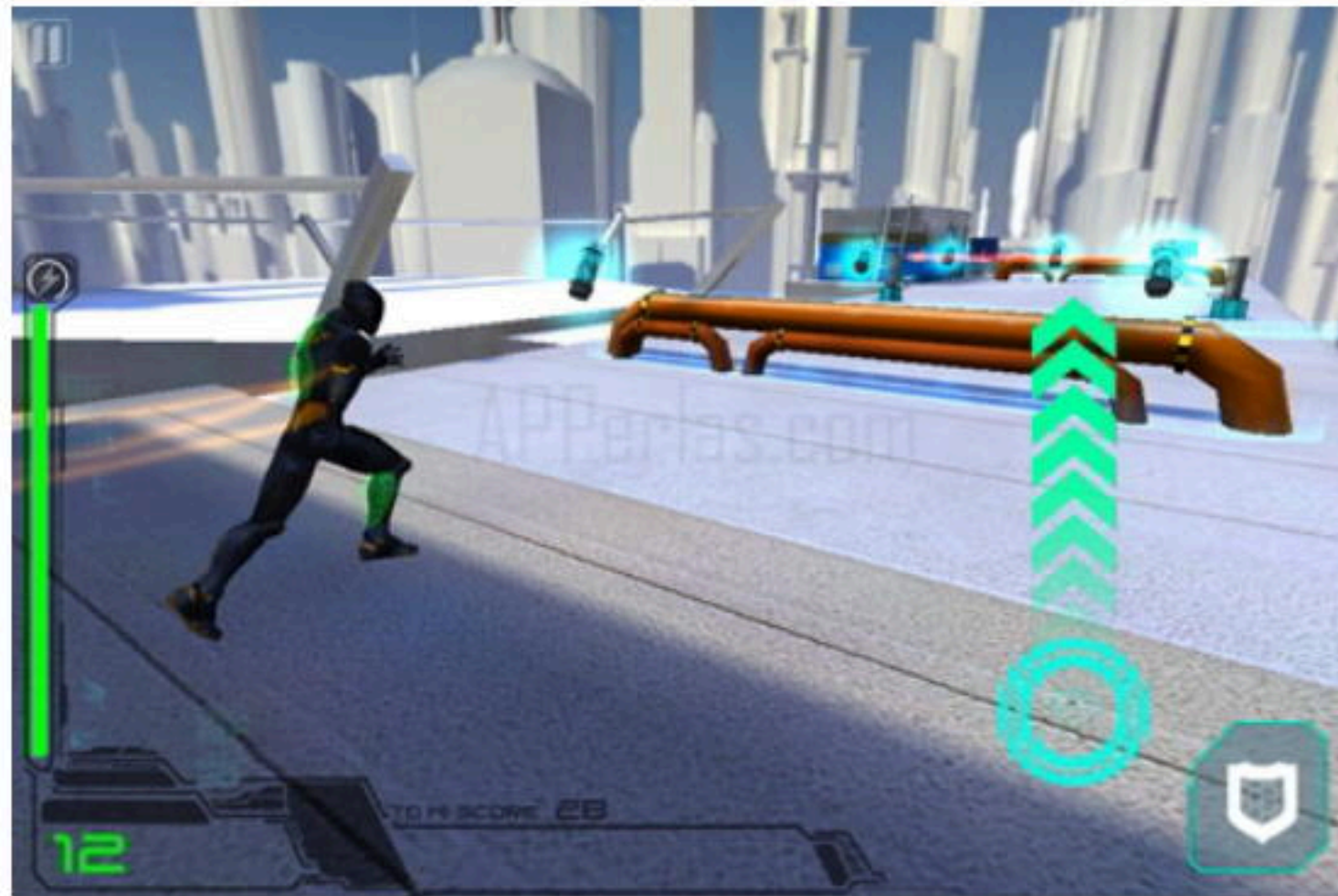
Once our brains were enough damaged, the prototyping phase started using the bubble pop engine made in Unity.

Finally the game was played to test all the functionalities and experiment it in real time.



Runbot -Runner Game-

A runner game in which your reflexes will be bomb proved. For this project I had to make an analysis of the different features and loops that had to be upgraded. I had to approach different disciplines for this game: from coding the new features designed, fixing the code of some shaders, balance the whole game in order to adapt it to the new features introduced in the gameplay, create new FX to give a new and brighter polish to the game and UI animations.



Xtreme Racing 2

A racing game with 3 different modalities: RC cars, Speed Boats and 4x4 RC cars. For this game, a full balance had be made: physics were upgraded changing all game controller behaviors. In this project I was focused in coding the car improvement section of the game, re-designing levels and tutorials and creating new FX for the game.



The Store

The place in which the progress of the game is mainly affected, improving the different parts of your car in order to defy more aggressive and difficult races.



Here I had to re-balance the improvements of the car in order to change the curve of difficulty inside the game. All this changes were made in an Excel file with a special format that could be imported into Unity to take all the data.



The Races

Different races with different difficulties that I had to re-balance due to physics upgrades inside the engine.



My main task here was re-designing game levels and balance the difficulty of the cars you had to defeat.

As polishing tasks, I had to introduce new FX system.

Skull Towers

A tower defense game in which your main objective is to defeat a bunch of enemy hordes before they reach your position.

I had different challenges for this game: Designing new Core Loops to increase the retention and implement them by code. Create a new system for the store. Create new FX systems for many different features of the game: mainly weapons. Balance all the levels in order to get a balanced difficulty curve.



Skull Towers

Balancing the Game Progress

The main challenge of this game was establishing a new game progress that had to be balanced due to a new reward system that had to be implemented.



There were a set of weapons and towers to be used for each level.

For this set of elements, a progress was established in order to unlock these kind of features in the appropriate moment in the game.



Weapons can be taken before each level.

The weapons to be set for each level was determined by the balance established for the progression of the game.



Inside each level, players can construct a type of tower depending on the type of altar.

The constructions had its own unlock progression, predetermined by the balance established.

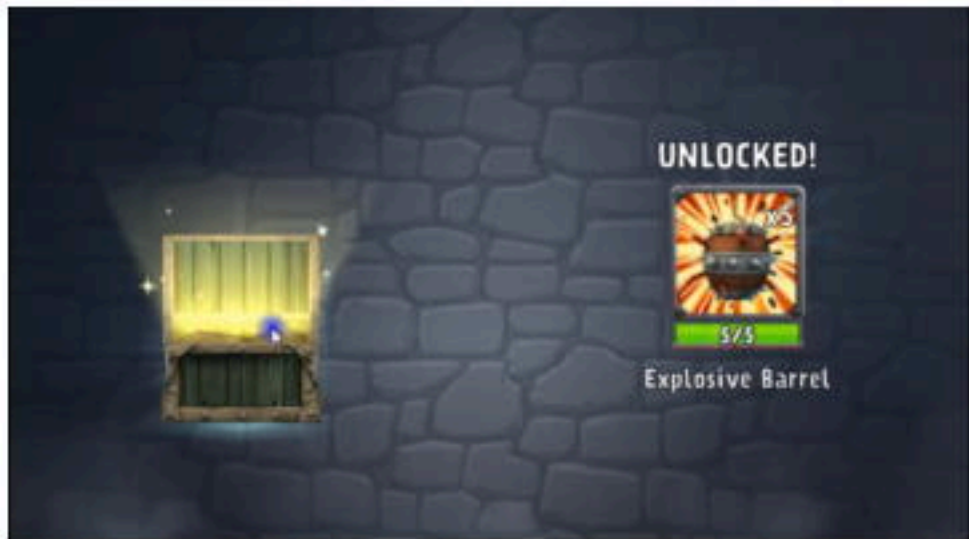
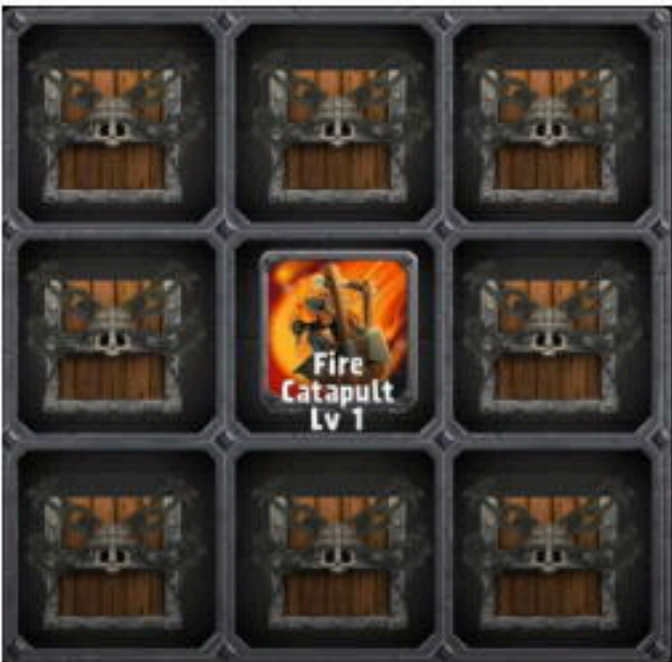


The balance of each element and the progression were tested playing different levels and combining the unlocked items to see if they fit well enough so as not to have an unbalance experience inside the global progression curve.

Skull Towers

Reward Systems

A new reward system had to be designed and implemented. My main task was to balance it and then code all the system to fit with the progression of the game.



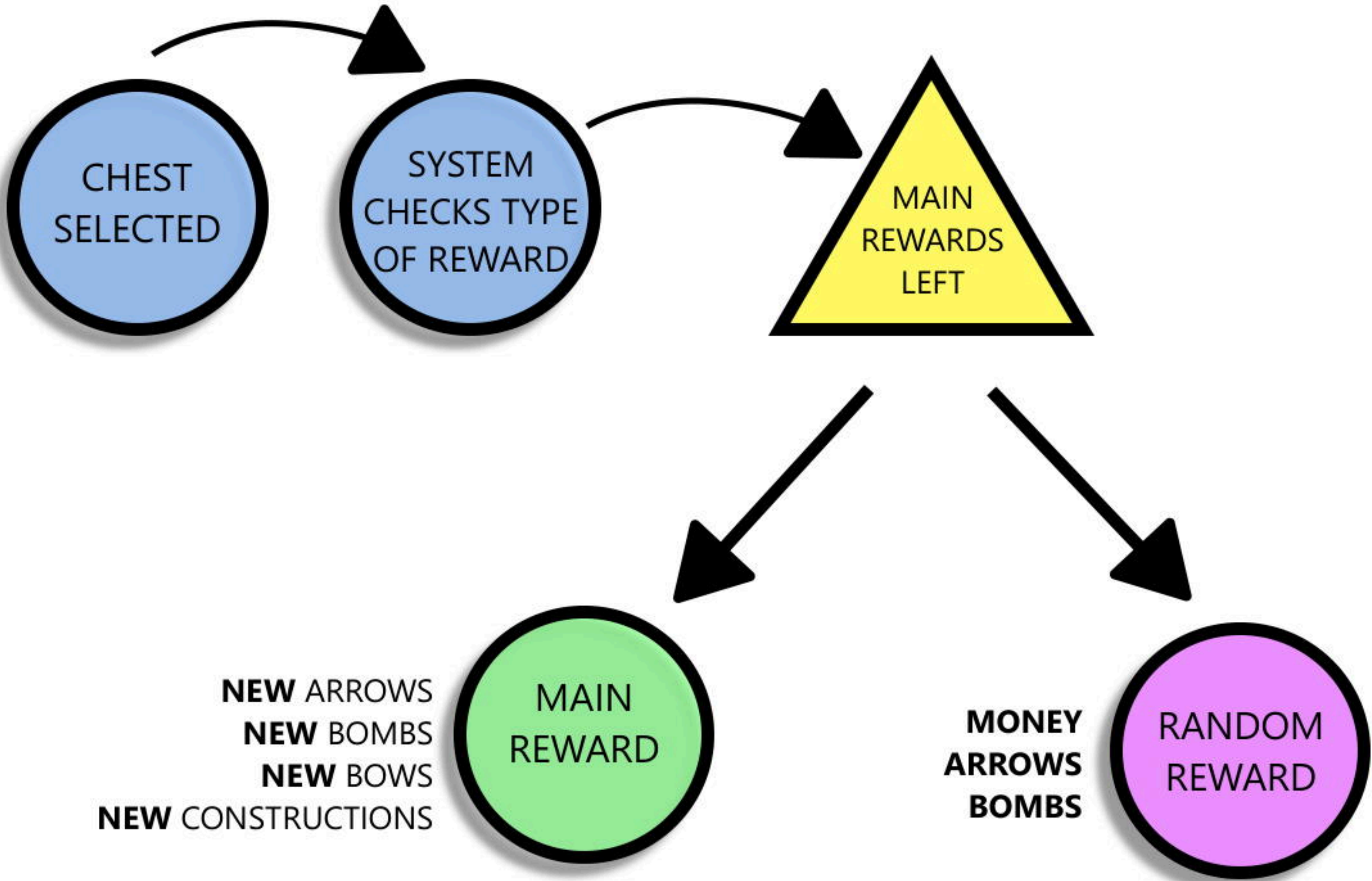
After each level, players had the possibility of unlocking a fixed set of chests depending on the number of keys obtained.

On this screen, the weapons and towers to be given were pre-set due to the curve of progression established.

In this case there are 2 type of rewards:

Main Rewards: the rewards to be unlocked to push the progression.

Random rewards: When there are no more Main Rewards to be given, the system gives randomly one of this kind of rewards.



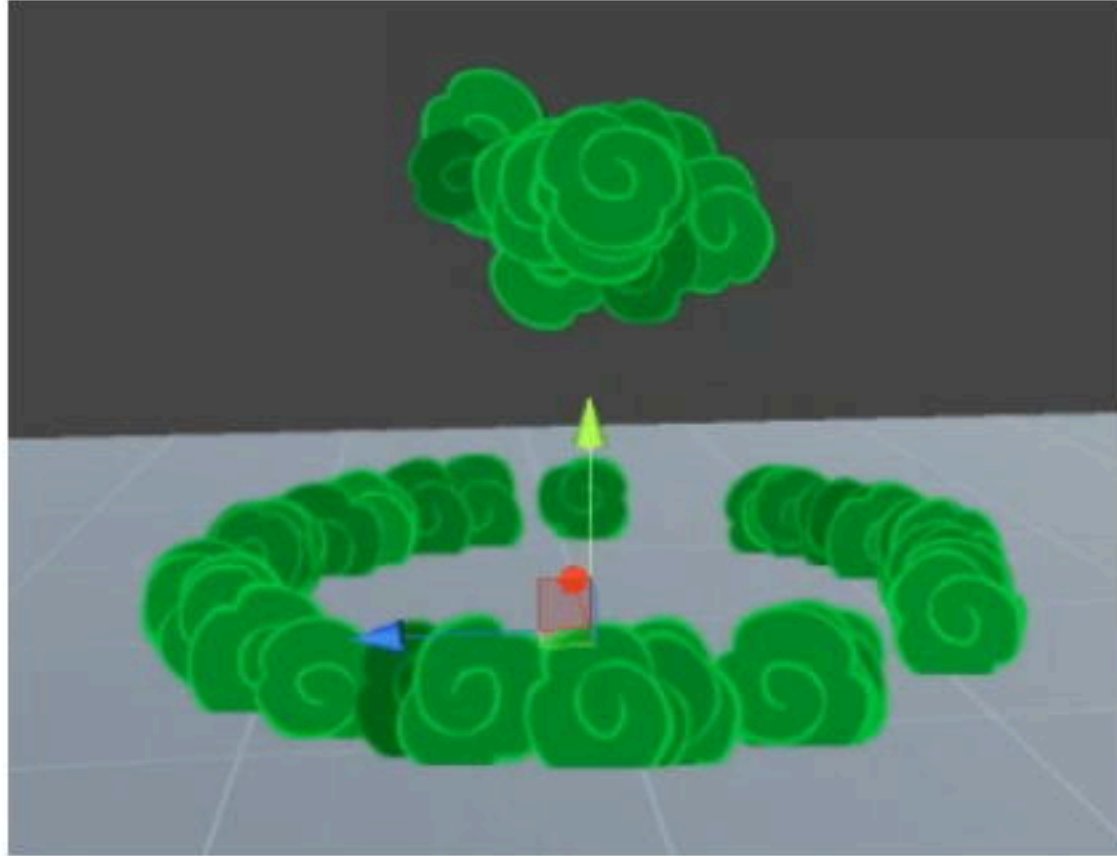
Skull Towers

FX (particle Systems)

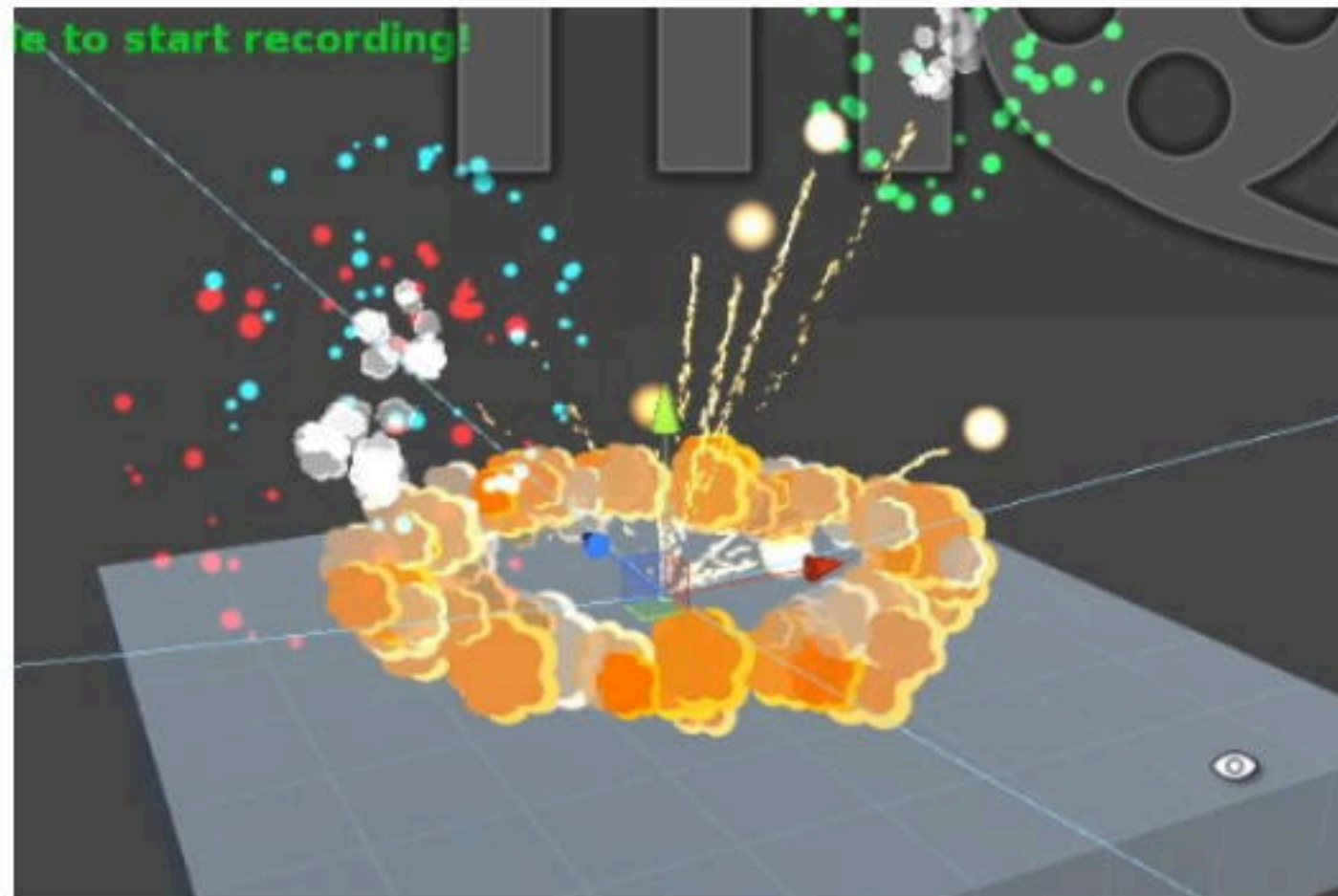
I had to design and implement different FX system done by myself. These FX systems were made fully inside Unity importing different elements from 3dsMax (models) or Photoshop(Textures).

The purpose of implementing these kind of systems was incrementing the final polish of the game.

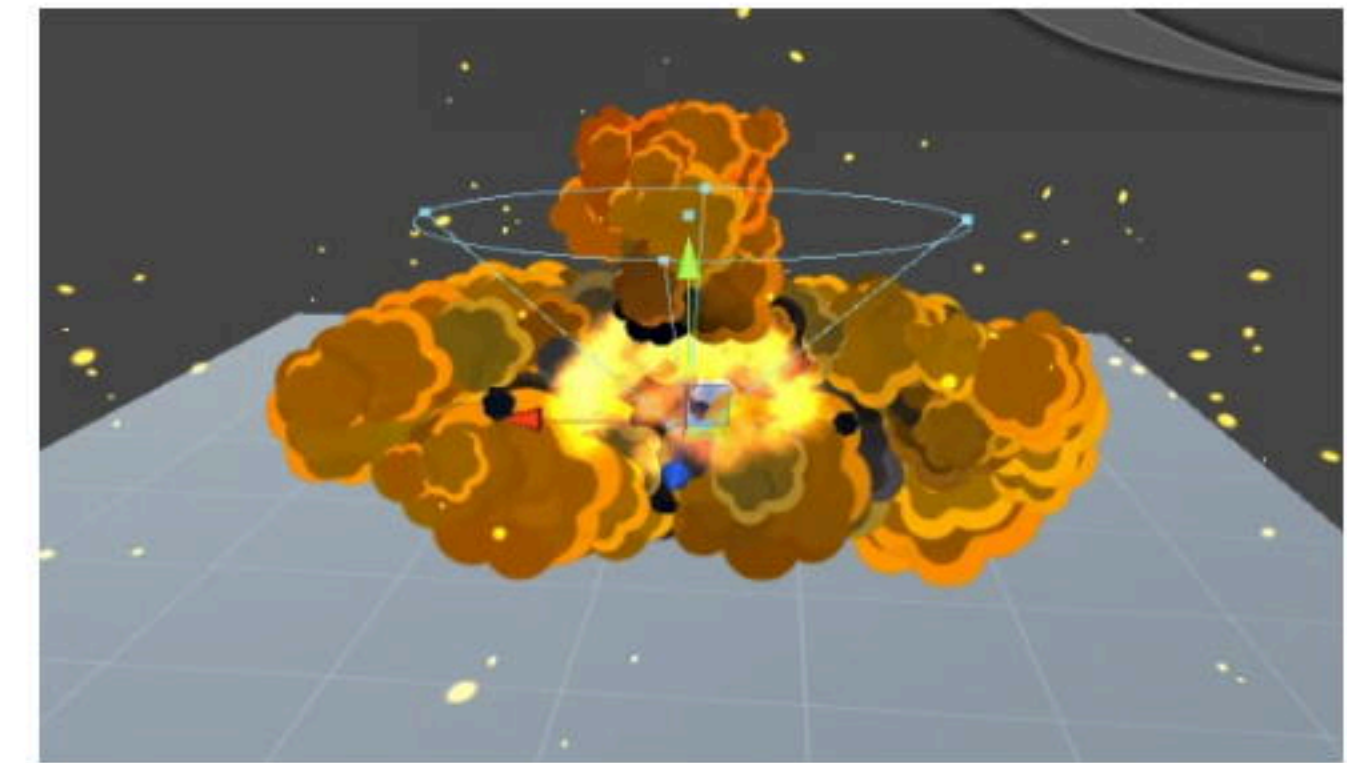
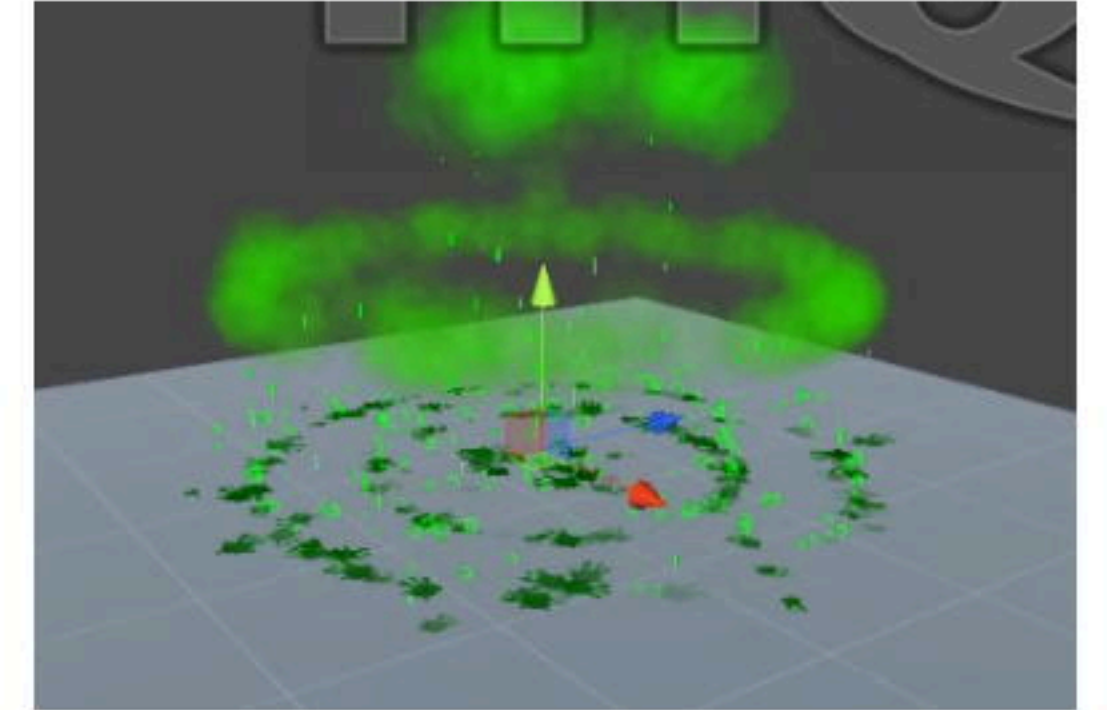
There is a Demo Reel attached in order to see the behavior of each Particle system. Here are attached some example images of the particle systems.



ENEMY DEAD



ARROWS



BOMBS

The main characteristics I have developed from this experience are: **diversity** and **versatility**.

Working for many different games genres at the same time and performing different roles, gave me enough **expansion** to develop myself and be **prepared** to work for any type of game.

The job environment wasn't the best one, but the great team I worked with, taught me that the **union** and **illusion** for what you **passion** for, is the engine to the develop yourself and the rest of the team as one.

Genera Games
CONCLUSIONS

TWS Inventors Of Play

Game Designer

Overview

TWS inventors of play is a game company focused in delivering Casino games for different web platforms through different markets around the world.

A team known as Creative and the one I was part of, had the responsibility of delivering **board** games and **video slot** games for the different clients we worked for.

My **mission** was **designing different** and **innovative** game experiences for video slots and board games.

From Videos Slot Games to Board Games, I had my own work **process** in order to deliver **concepts**, **documentations** and **prototypes** about each game we created. I will explain my own process through a Slot Game I developed for the Innovative Department of the company.

All the games I have developed inside the company are in a web page known as: **<https://www.bovada.lv>** You will need a VPN to enter inside it.

Games: Gold Rush Gus, 777 Deluxe, Cyberpunk City, Gods Of Luxor, Teen Patti, Andar Bahar, Wheel of Fortune, Cricket Legends, Zombie Soccer and many more...

Process



An idea breaks into my mind.



An amorphous thing is trying to get alive inside my brain.

The Behaviour of that thing is matched with other similar behaviours I research to find something coherent: science, nature other similar games...

ITERATION

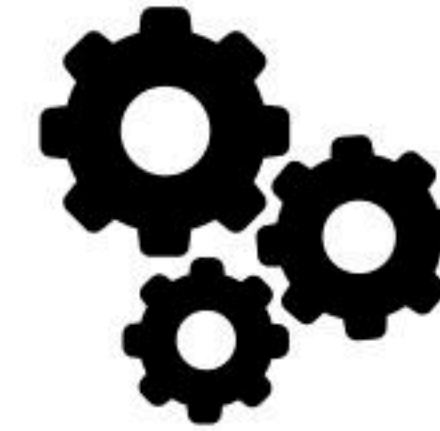


A black marker scribbles a white board trying to give form to the idea.

All the features are documented inside a GDD



The idea is shown to the mathematician and the backend team to see if it is possible to be prototyped mathematically



Game Concept is prototyped in Unity Engine.

Math are simulated directly in a console app in C#.

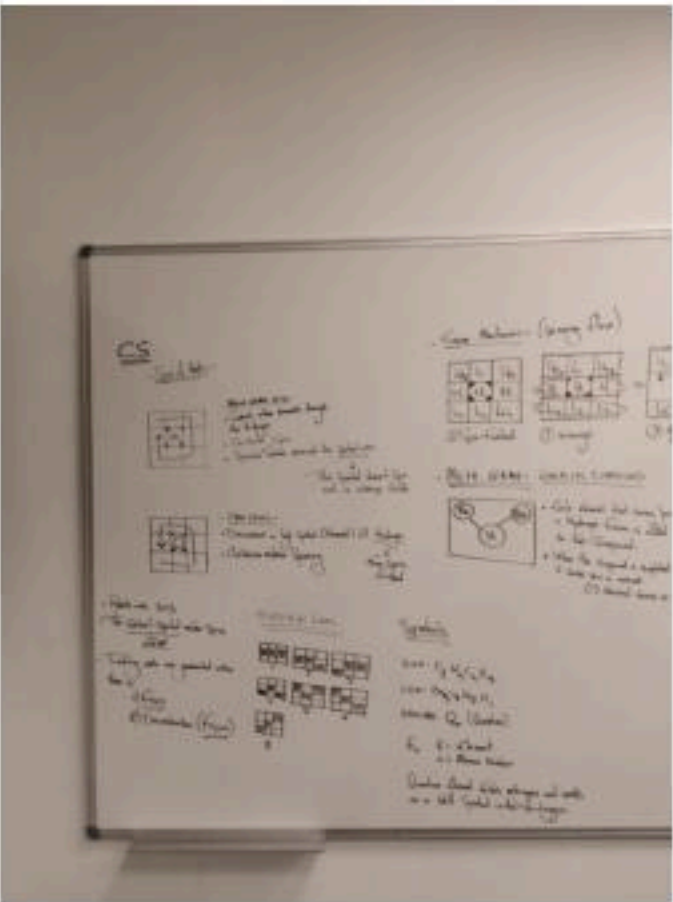
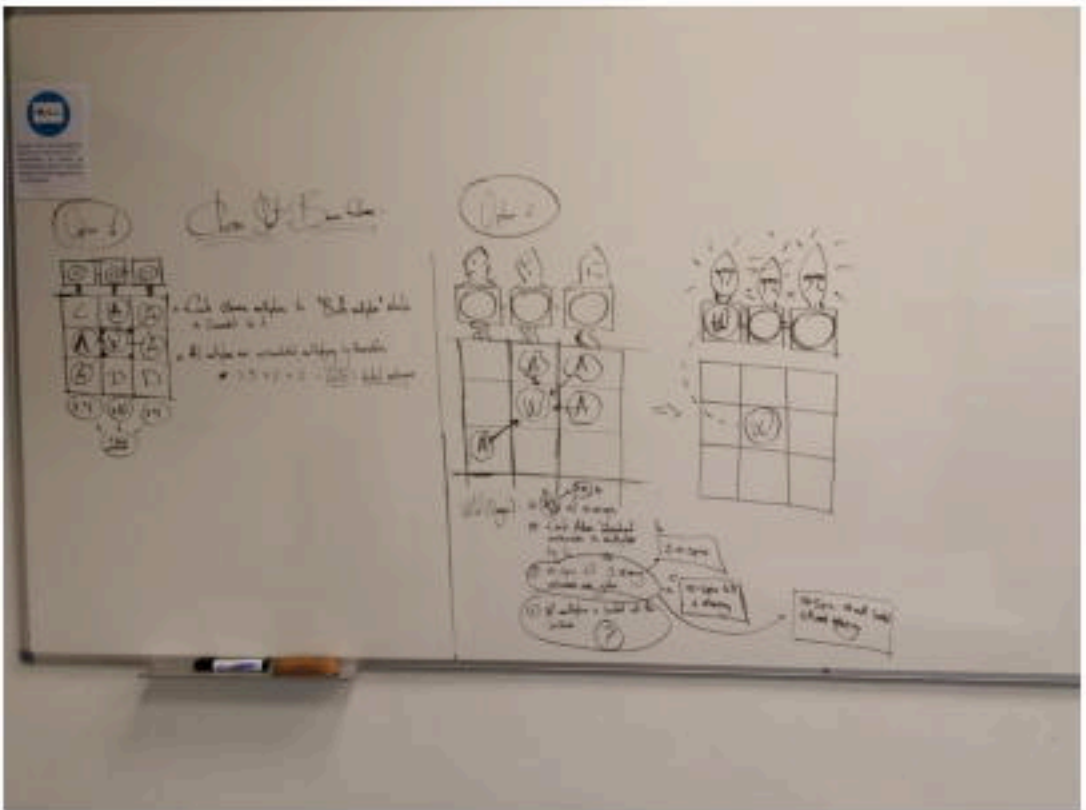
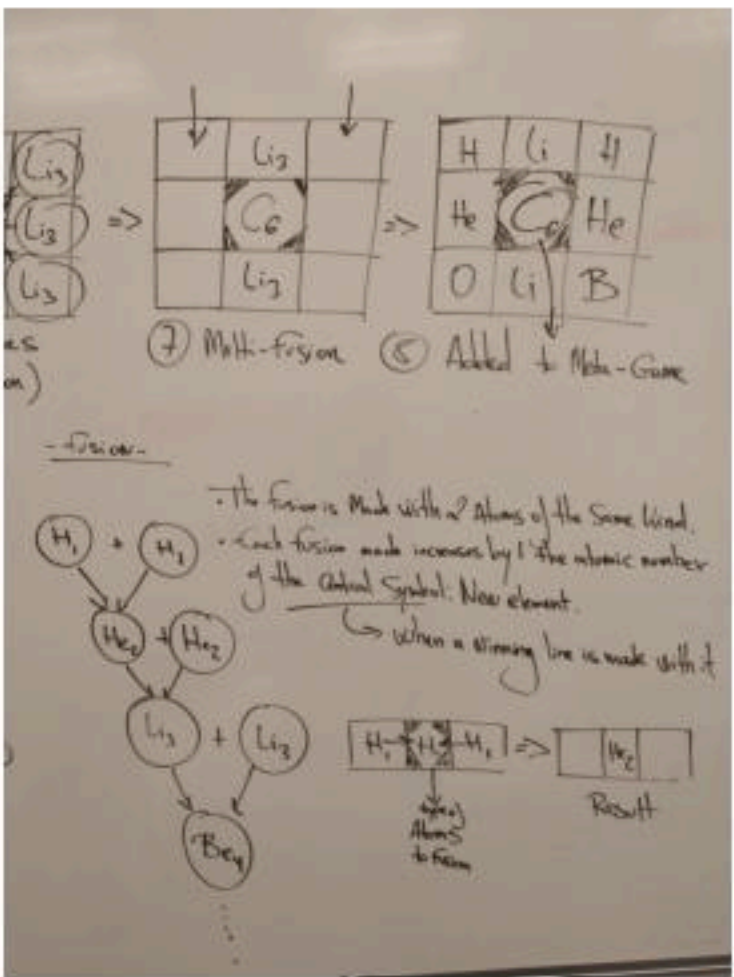
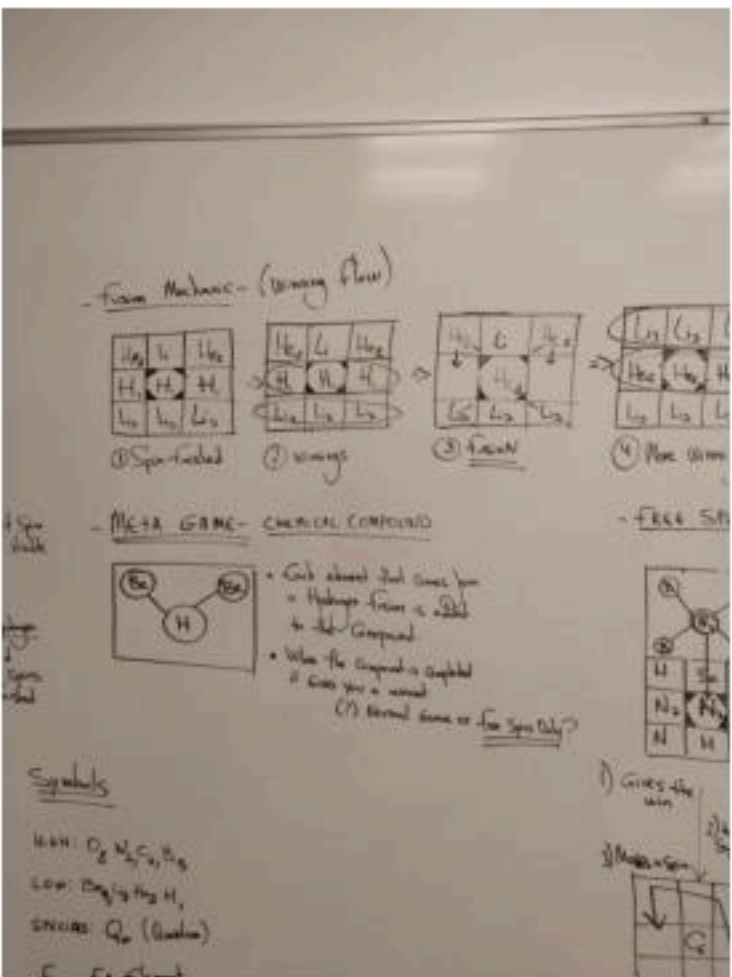
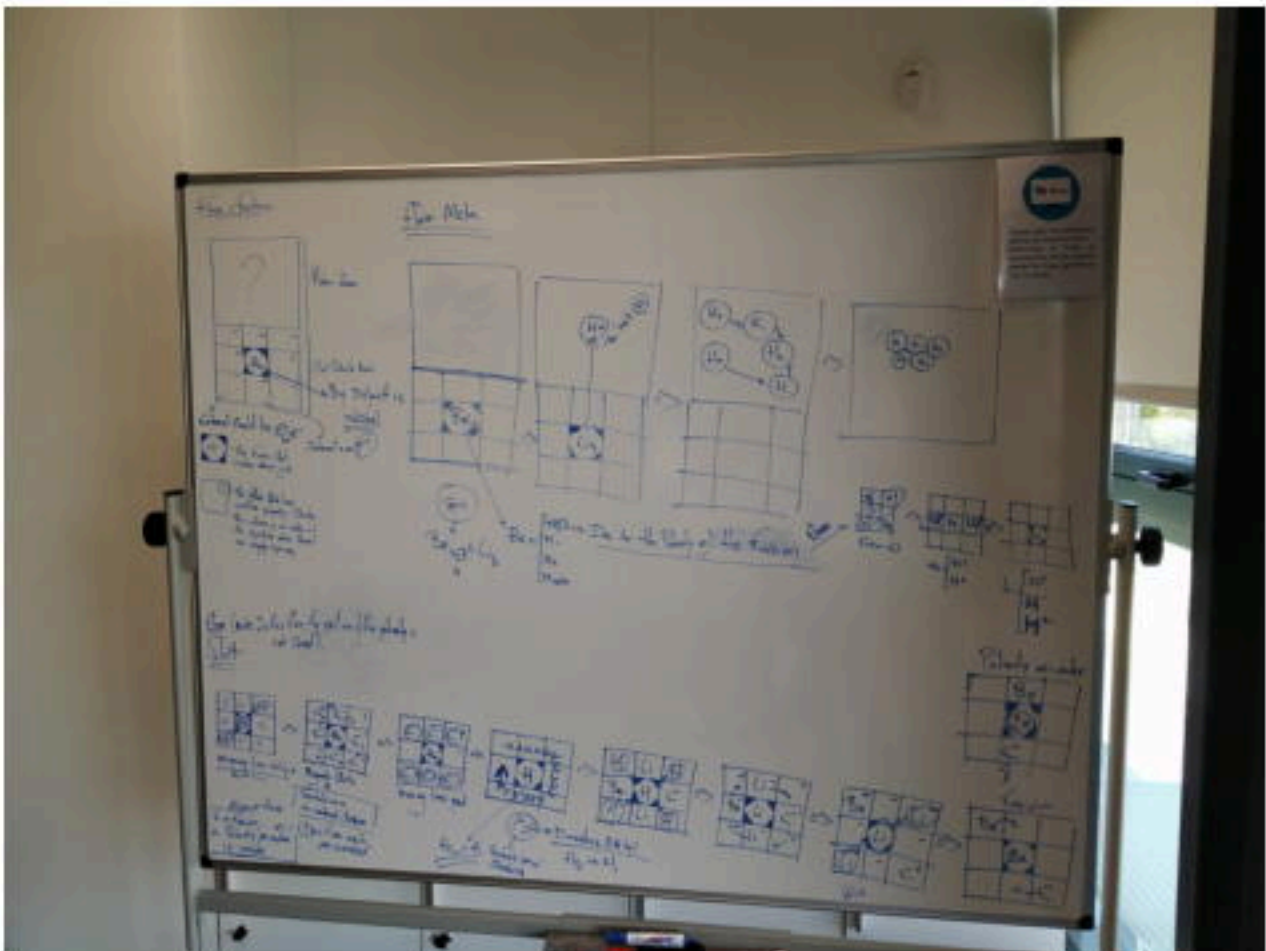
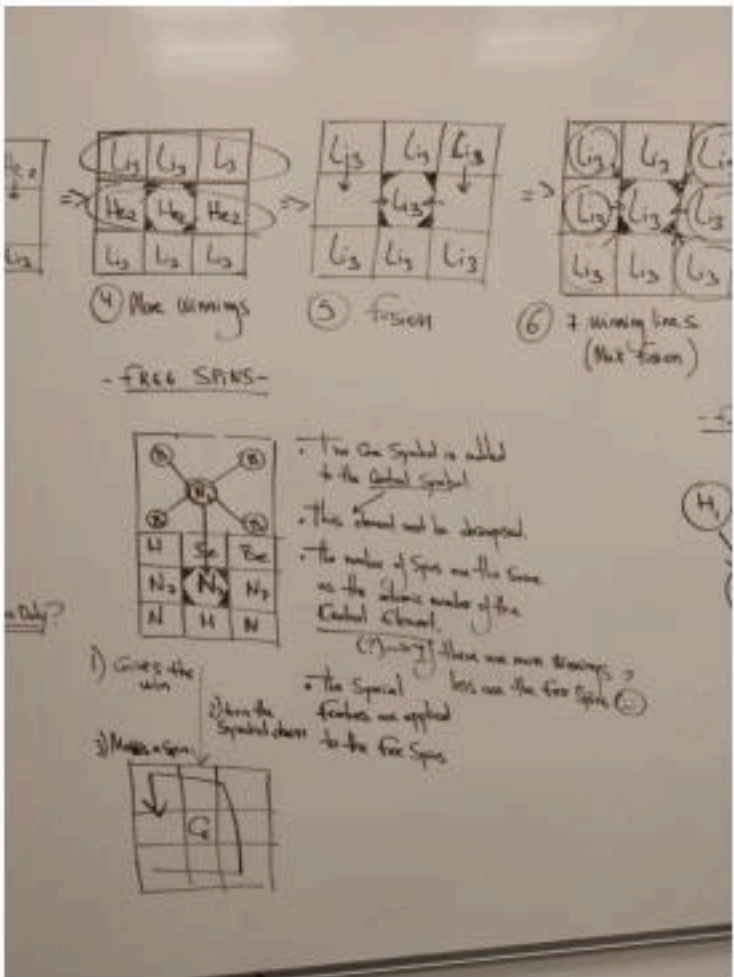
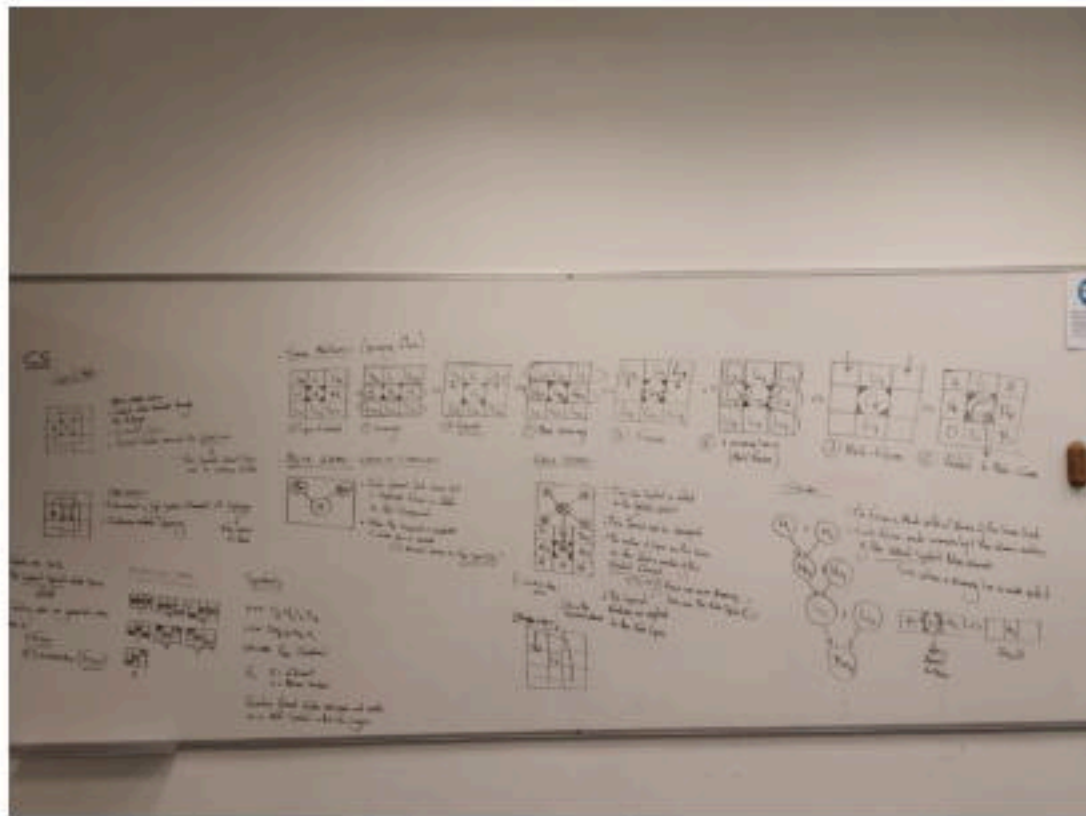


Game Designer and Mathematician analyze the game and set different test sessions to see if the game is feasible for players and can be balanced properly.

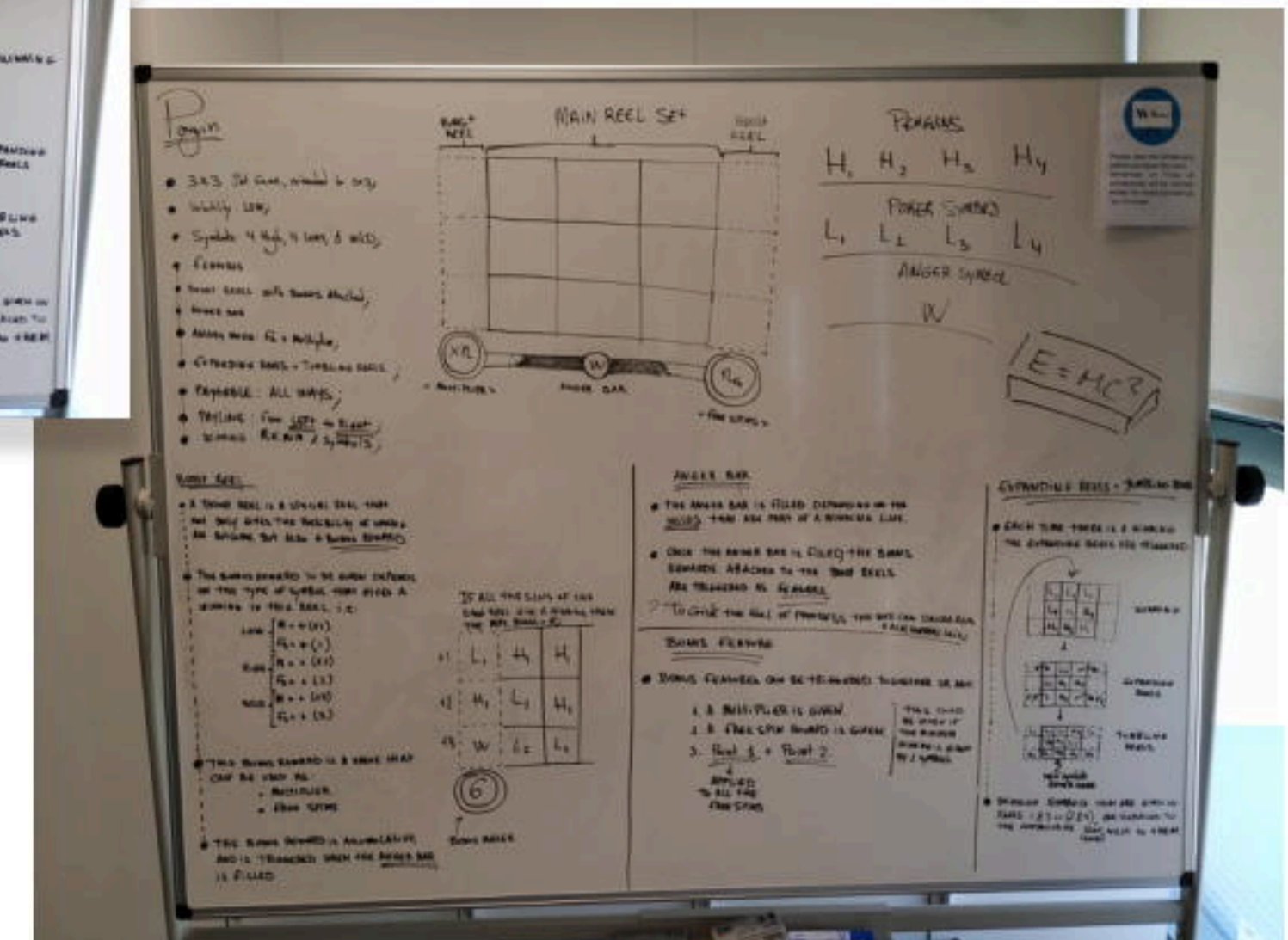
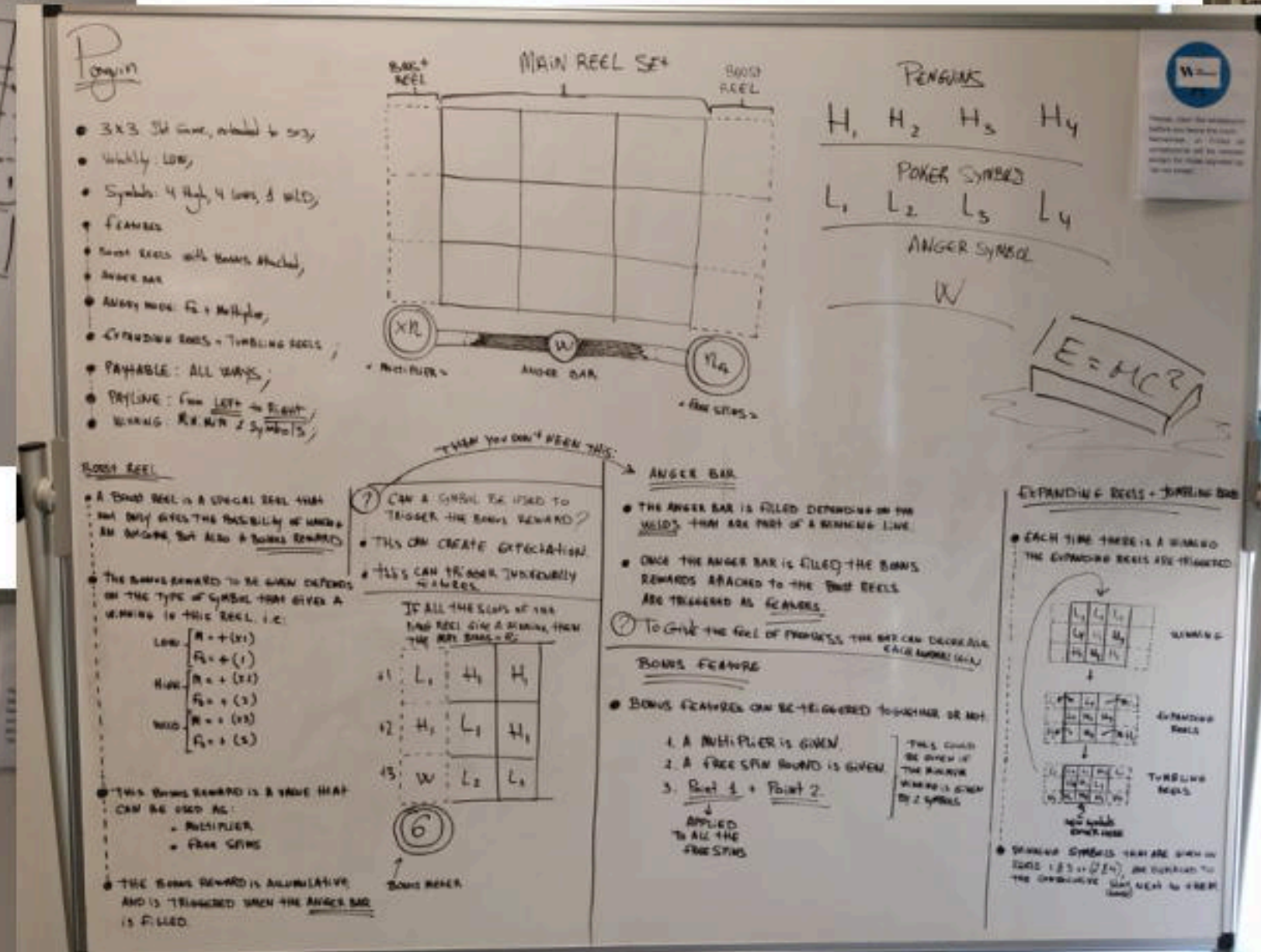
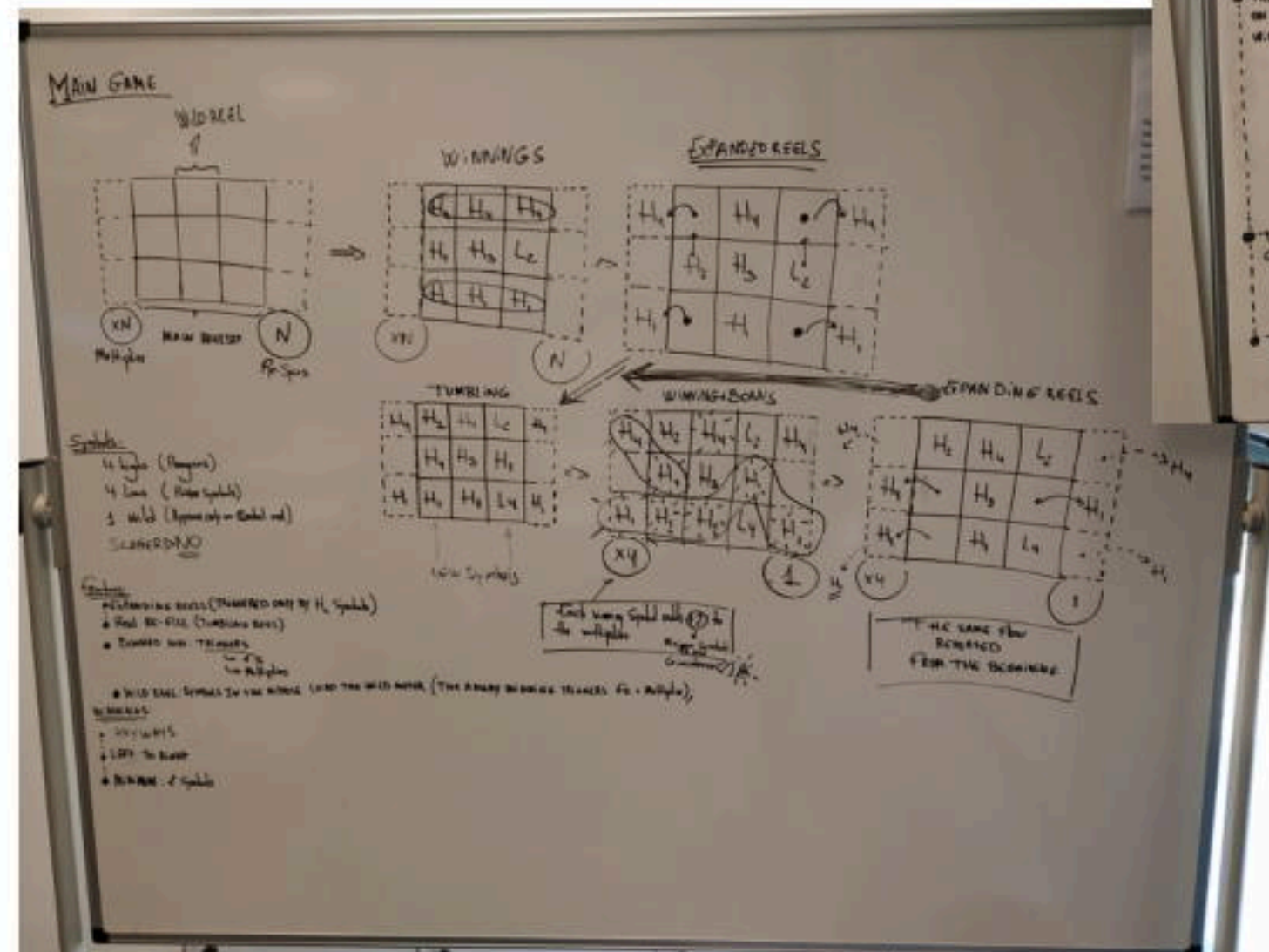
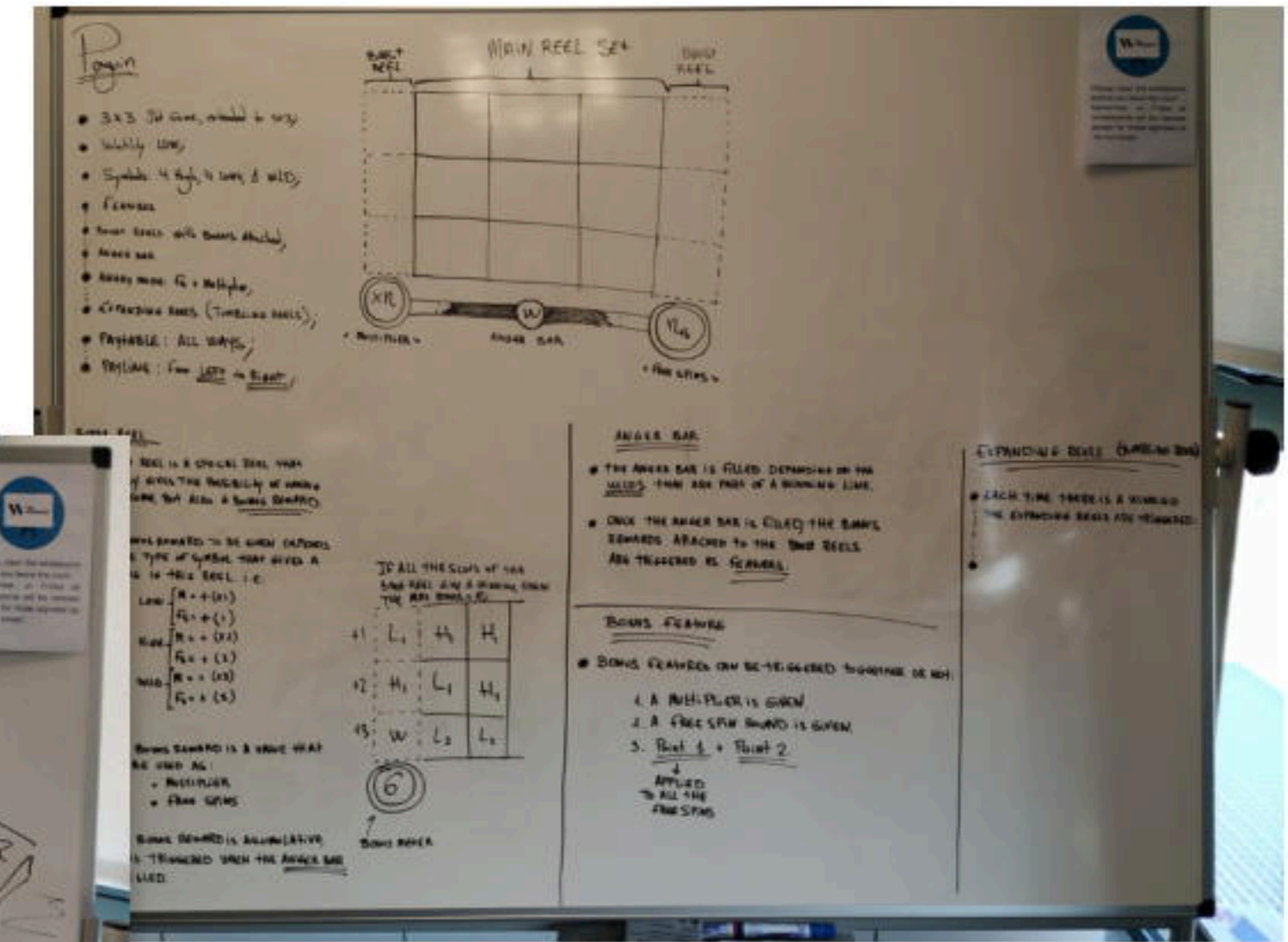
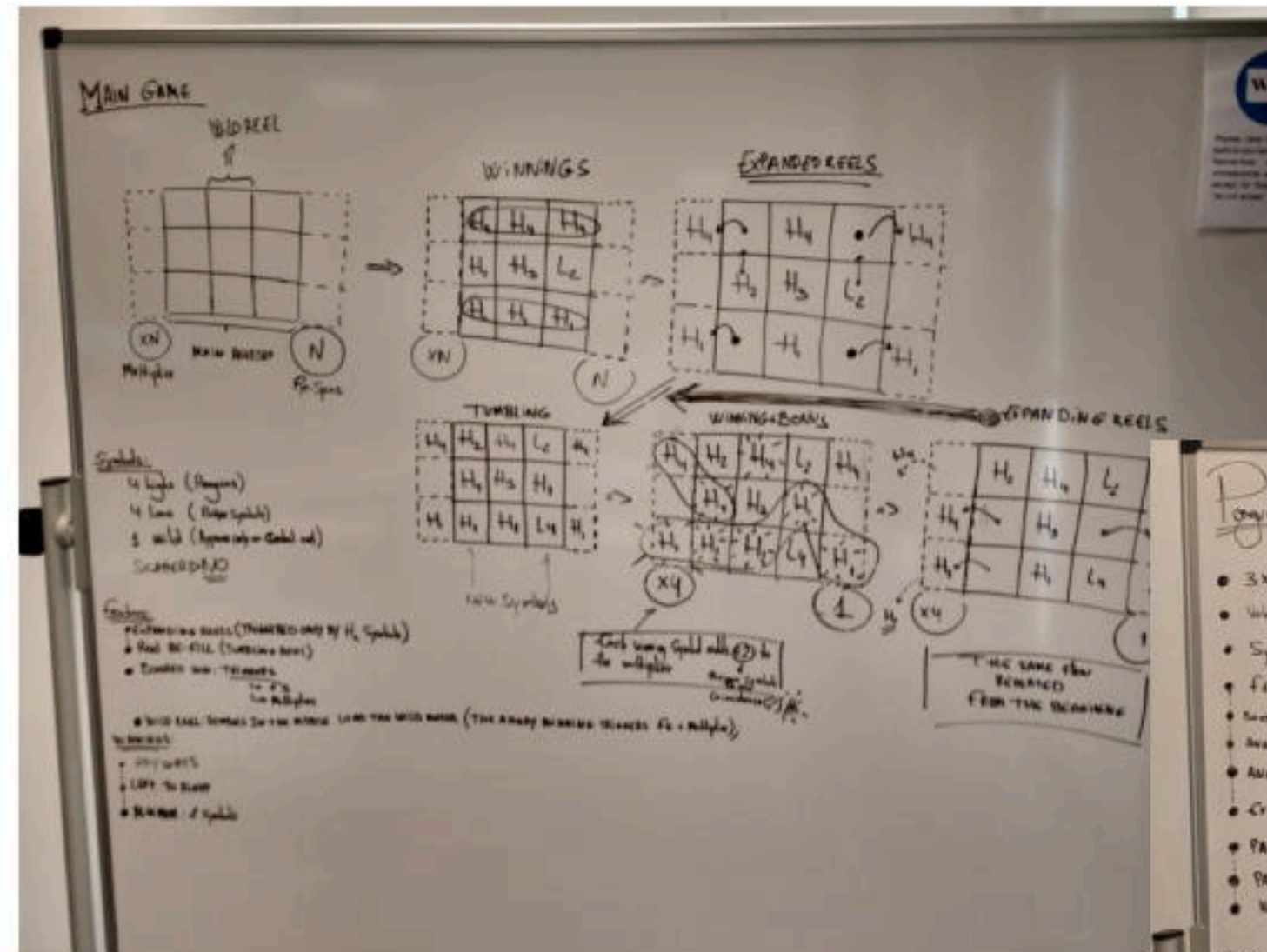
Quantic (Slot Game)



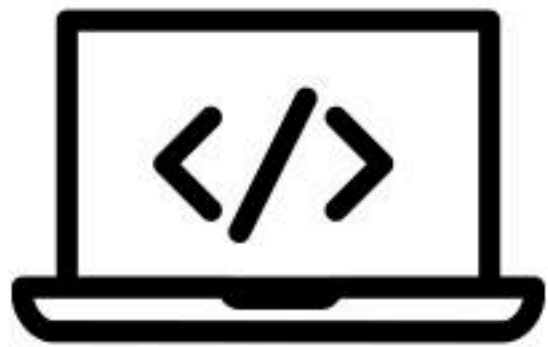
Different features are explored analyzing them on a white board.



Quantic (Slot Game)



Quantic (Slot Game)



After the Game Conceptualization phase finished, I coded all the backend in order to simulate the game and take a proper math profile.

```
[System.Flags]
public enum direction
{
    ALL = (1 << 0),
    LEFT = (1 << 1),
    RIGHT = (1 << 2),
    UP = (1 << 3),
    DOWN = (1 << 4),
    CENTRAL = (1 << 5) //Winning lines are paid only if the central symbol matches with the other symbols.
}

public abstract class MatrixReader : ScriptableObject
{
    public direction directionReader;

    public abstract List<BE_Payline> Reader(List<int> matrix, Vector2Int matrix_dimensions, List<BE_Symbol_SO> symbols_database);
}

/// <summary>
/// Read the current matrix and returns an Outcome
/// </summary>
/// <typeparam name="O">The Output parameter to be given by the matrix reader</typeparam>
public abstract class MatrixReader<O> : ScriptableObject
{
    public direction directionReader;

    public abstract O Reader(List<int> matrix, Vector2Int matrix_dimensions, List<BE_Symbol_SO> symbols_database);
}
```

```
if ((Mathf.FloorToInt(p / matrix_dimensions.x) - 1 == Mathf.FloorToInt(item.Value[i] / matrix_dimensions.x))
    || (Mathf.FloorToInt(p / matrix_dimensions.x) + 1 == Mathf.FloorToInt(item.Value[i] / matrix_dimensions.x)))
{
    if ((item.Value[i] - matrix_dimensions.x == p)
        || (item.Value[i] + matrix_dimensions.x == p)
        || ((item.Value[i] - (matrix_dimensions.x + 1)) == p)
        || ((item.Value[i] - (matrix_dimensions.x - 1)) == p)
        || ((item.Value[i] + (matrix_dimensions.x + 1)) == p)
        || ((item.Value[i] + (matrix_dimensions.x - 1)) == p))
    {
        if (!group.Contains(item.Value[i])) { group.Add(item.Value[i]); }

        if (!group.Contains(p)) { group.Add(p); }
    }
}
```

```
#region Resources
//All the Backend data needed to make the simulation
public BE_Data be_Data;

//The current rtp in the game
public Backend_RTP_SO rtpData;

//Matrix Reader
public MatrixReader reader;
public bool isReSpin = false;
public bool isFusion = false;
public bool simulation = false;
#region Simulation Data
[Header("Global RTP Contribution")]
public float rtp = 0F;
public float baseContribution = 0F;
public float bonusRTP = 0F;

[Header("Frequencies")]
public float winningHits = 0F;
public float hitRate = 0F;
public float reSpinsFrequency = 0F;
public float bonusFrequency_1_IN = 0F;
public float fusionFrequency = 0F;

[Header("Winings")]
public float total_win = 0F;
public float baseGameTotalWin = 0F;
public float totalBonusWin = 0F;
public float totalRespins = 0F;
public float numberOfBonusGiven = 0F;
public float totalFusions = 0F;

[Header("Spins")]
public int spins = 0;
[HideInInspector]
public int maxSpins = 10000;

#endregion
```

```
if (centralSlot.runtimeValue.symbol_id == 10)
{
    _totalHits++; //Increase the win when appears
    _wildAppearRate = 1F / (_totalHits / _totalSpins);
    globalHitRate += _wildAppearRate;
}

//Save paylines
_paylines = new List<BE_Payline>(reader.Reader(currentSpin, be_Data.be_data_persistent.matrix_dimensions, be_Data.be_data_persistent.be_symbols));

//Set the winning lines
if (_paylines.Count > 0)
{
    be_Data.be_data_persistent.matrix.SetWinningSymbols(_paylines);
    _totalOutcome += _paylines.Select(p => p.outcome).Sum() * _initBonusMultiplier;
    _spinWin += _paylines.Select(p => p.outcome).Sum() * _initBonusMultiplier;
    _winningHits++;
}

//Check if there is any fusion
Matrix outFusion = SlotFeatures.Fusion.Trigger(be_Data.be_data_persistent.matrix);

let alphaSymbolsMatrix = 0;
let omegaSymbolsMatrix = 0;

if (outFusion.slots.FindAll(slot => slot.runtimeValue.symbol_id == (int)slotType.ALPHA).Count > 0)
{
    outFusion.slots.ForEach(slot =>
    {
        if (slot.runtimeValue.symbol_id == (int)slotType.ALPHA)
        {
            alphaSymbolsMatrix++;
            _freeSpins++;
        }

        if (slot.runtimeValue.symbol_id == (int)slotType.OMEGA)
        {
            omegaSymbolsMatrix++;
            _multiplier++;
        }
    });
}
```

```
if (!isFusion)
{
    //Generate a Spin
    currentSpin = new List<int>(Spin());
}

//Save paylines
_paylines = new List<BE_Payline>(reader.Reader(currentSpin, be_Data.be_data_persistent.matrix_dimensions, be_Data.be_data_persistent.be_symbols));

//Set the winning lines
if (_paylines.Count > 0)
{
    be_Data.be_data_persistent.matrix.SetWinningSymbols(_paylines);
    DisplayInfo(_paylines);
    total_win += _paylines.Select(p => p.outcome).Sum();
    baseGameTotalWin += _paylines.Select(p => p.outcome).Sum();
    auxSpinWin += _paylines.Select(p => p.outcome).Sum();
}

//Check if there is any fusion
List<int> outFusion = SlotFeatures.Fusion.Trigger(be_Data.be_data_persistent.matrix);

if (outFusion != null && outFusion.Count > 0)
{
    currentSpin.Clear();
    currentSpin = new List<int>(outFusion);
    be_Data.be_data_persistent.matrix.ResetWinningSymbols(); //Reset winning symbols
    _paylines.Clear();
    isFusion = true;
    totalFusions++;
    fusionFrequency = 1F / (totalFusions / spins);
}
```


Quantic (Slot Game)



Coding all the Backend helped me to Balance the game through Unity editor with no need of Excel files.

With this method I could make the adjustments needed in order to balance the game and at the same time, I could simulate it inside Unity to get the appropriate data.

Script: BE_Symbol_SO

▼ Init Value

Id

7

Name

H2

Weight

5

Is Main

☒

Paytable

3

Value

3

Mg_probability

0.04672897

Bn_probability

0.04273504

► Runtime Value

Be_Data

BE_Data (BE_Data)

Script: BE_Symbol_SO

▼ Init Value

Id

1

Name

L4

Weight

30

Is Main

☒

Paytable

3

Value

0.2

Mg_probability

0.2727273

Bn_probability

0.2702703

► Runtime Value

Be_Data

BE_Data (BE_Data)

BALANCING

Global RTP Contribution	
Rtp	0.9768999
Base Contribution	0.6612856
Bonus RTP	0.3156919
Frequencies	
Winning Hits	47525
Hit Rate	1.578117
Re Spins Frequency	0
Bonus Frequency_1_IN	79.19113
Fusion Frequency	0.4719827
Winnings	
Total_win	73267.79
Base Game Total Win	49596.72
Total Bonus Win	23675
Total Respins	0
Number Of Bonus Given	947
Total Fusions	158902
Spins	
Spins	75000
Bonus Outcome	25
Simulate	

SIMULATION DATA



Meanwhile, all the art was made by the concept artist who I worked with.

Meet Paolo Vicenci: <https://www.linkedin.com/in/paolo-vicenzi-56b67149>



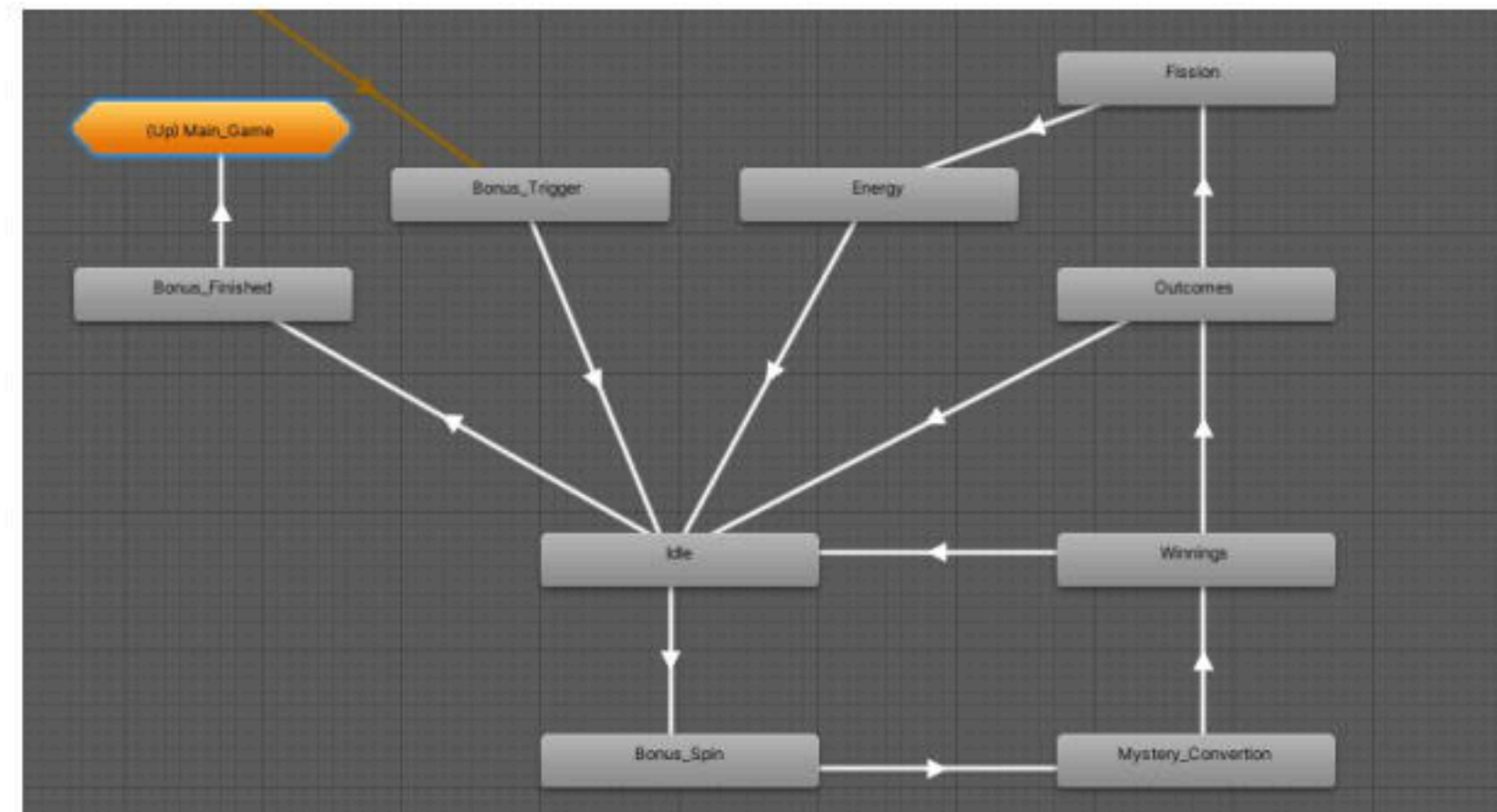
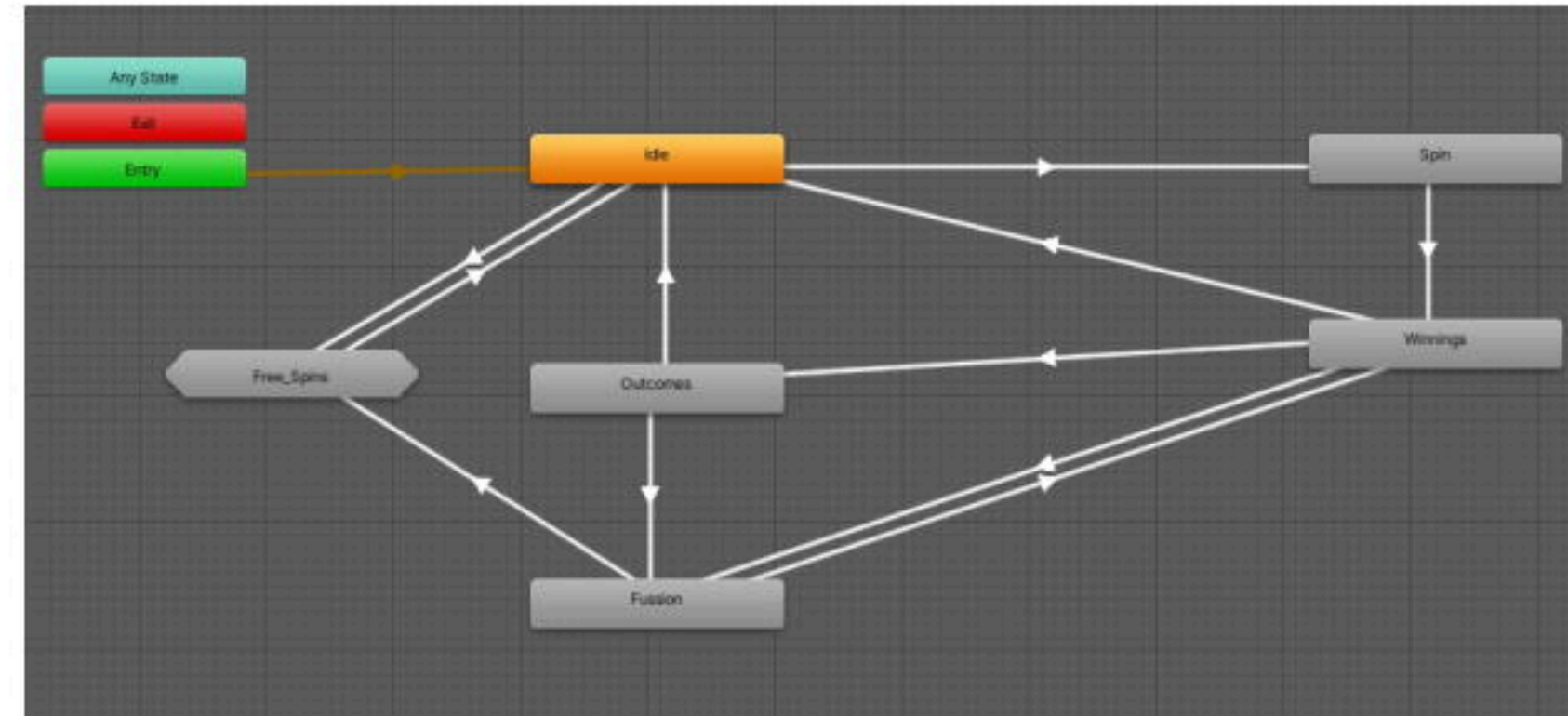
Quantic (Slot Game)



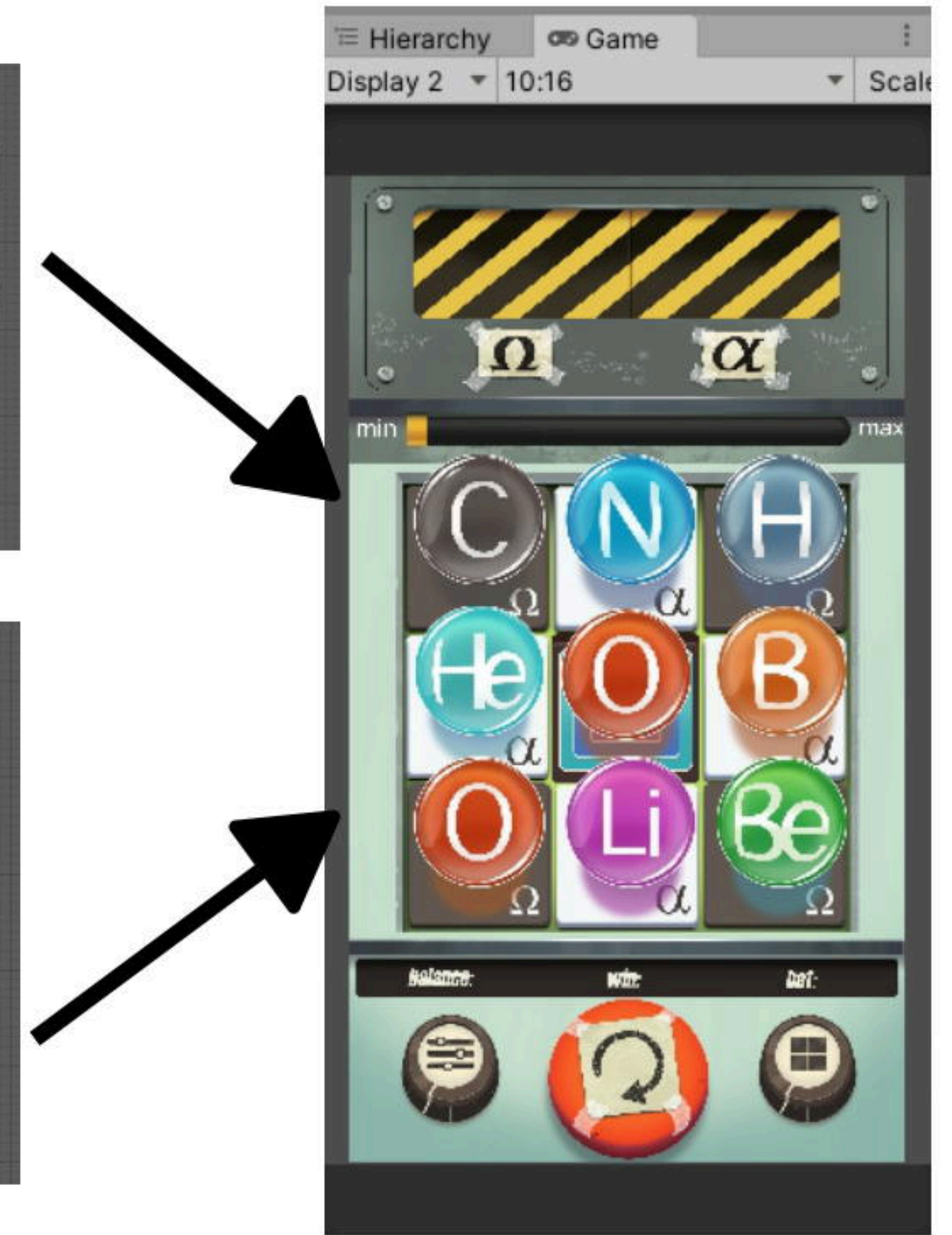


When all the assets were finished, it were substitutes over the placeholders of the frontend side.

The frontend had its own state machine made with the animator in order to control each state of the game step by step.



Quantic (Slot Game)



Learning from **slot games** is an experience full of **creative strives**. The **limits** you have in order to make something so simple but at the same time original, is a challenge I do recommend to anyone that wants to expand his/her knowledge in the world of game design.

Technically is a strong experience: I acquired a good knowledge about **probabilities** and how to **simulate** them in order to get a game based in numeric data; the level of **abstraction** is really high in this process.

The team I worked with is probably the best team and people I have met at this time: **communication, ideas** and **collaboration** were 3 pillars **naturally** performed by all the team members.

TWS Inventors of Play
CONCLUSIONS

EA Sports

Game Designer

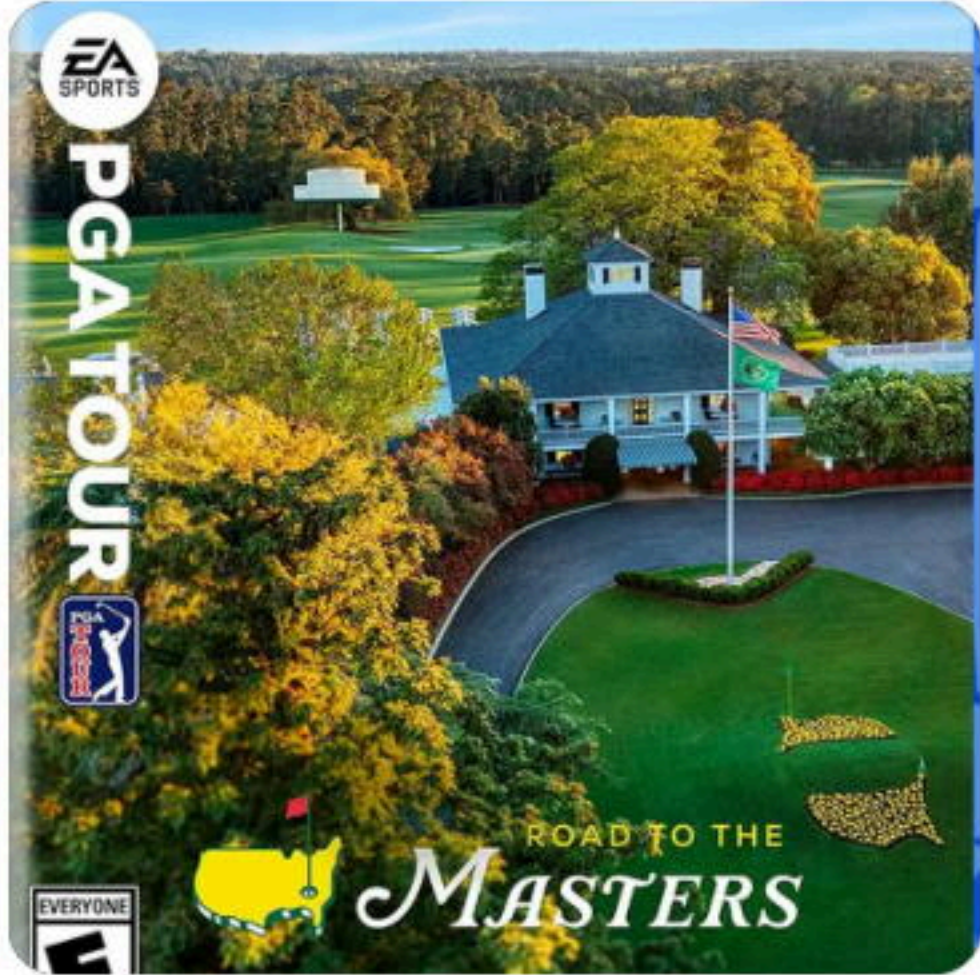
Overview

At EA SPORTS Madrid, I worked across the full production cycle of EA SPORTS PGA TOUR and the Superstar experience in Madden NFL 25, Madden NFL 26 and Madden NFL 27.

My PGA TOUR work was delivered in a fully remote setup, covering progression, quests, tournaments and live-service systems. In Madden, my focus expanded into Superstar archetypes and abilities, relationship-driven progression, live events and seasonal content.

The common thread was systems design: connecting player goals, progression, rewards, content pipelines and production constraints into features that could be built, tested, shipped and supported.

Games



EA SPORTS
PGA TOUR



Madden NFL 25



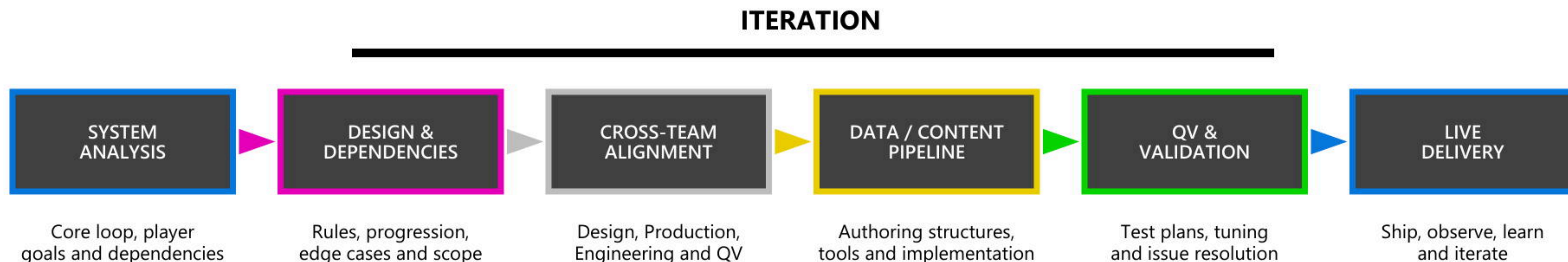
Madden NFL 26



Madden NFL 27

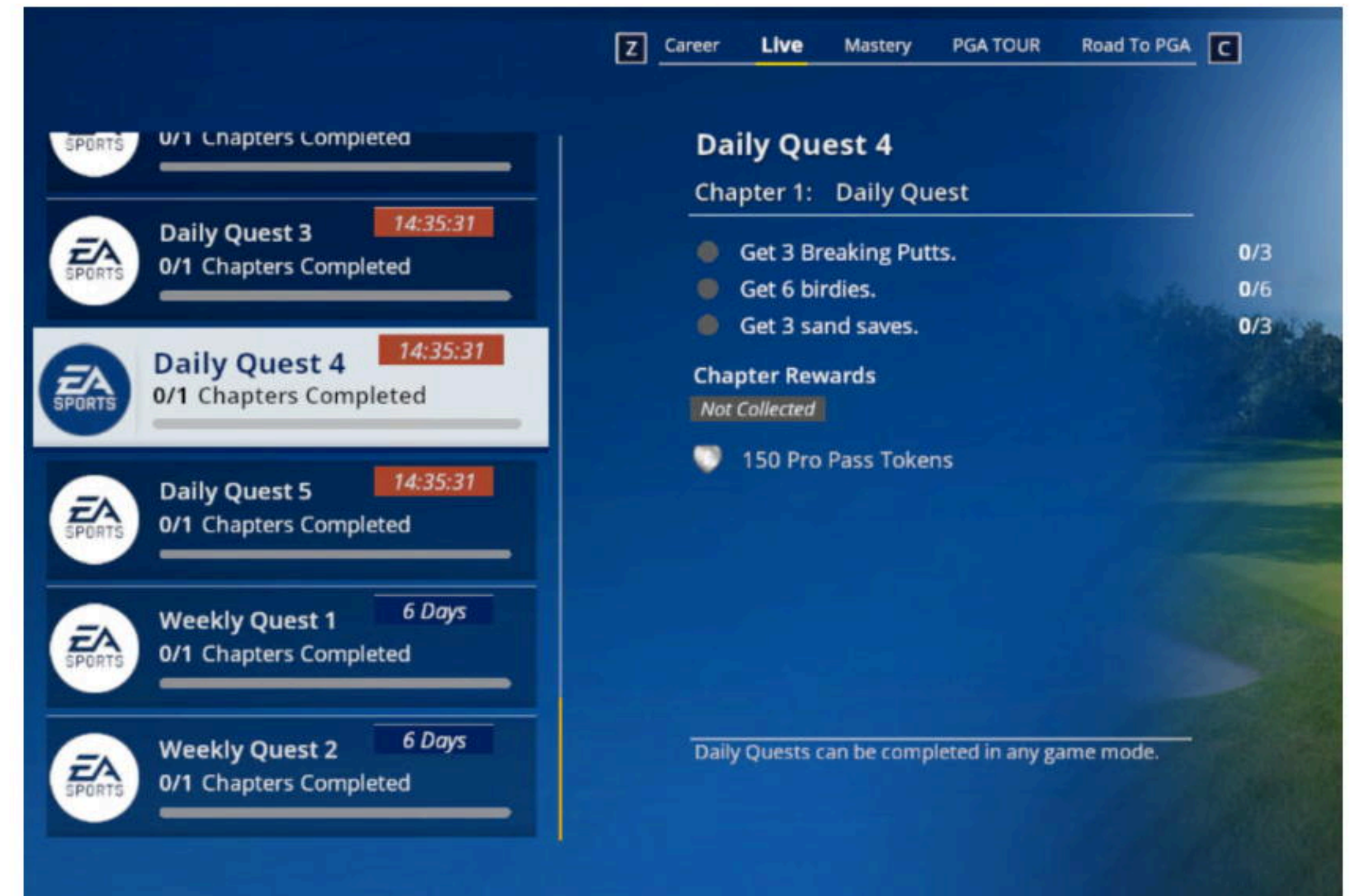
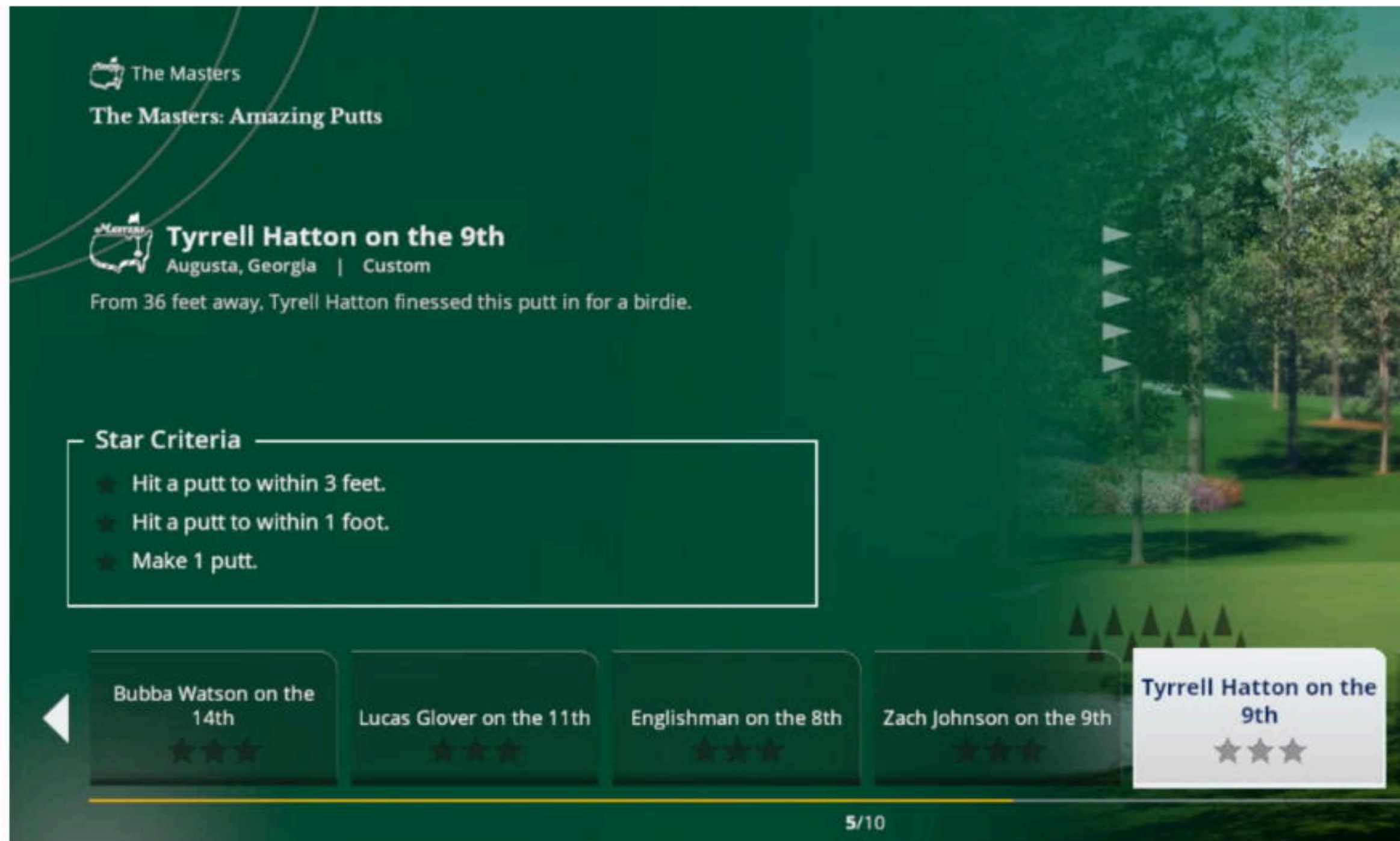
Process

My work at EA SPORTS connected system design with production: understanding the player loop, mapping dependencies, aligning with partner teams, authoring data and content, validating the experience and iterating through release and live service.



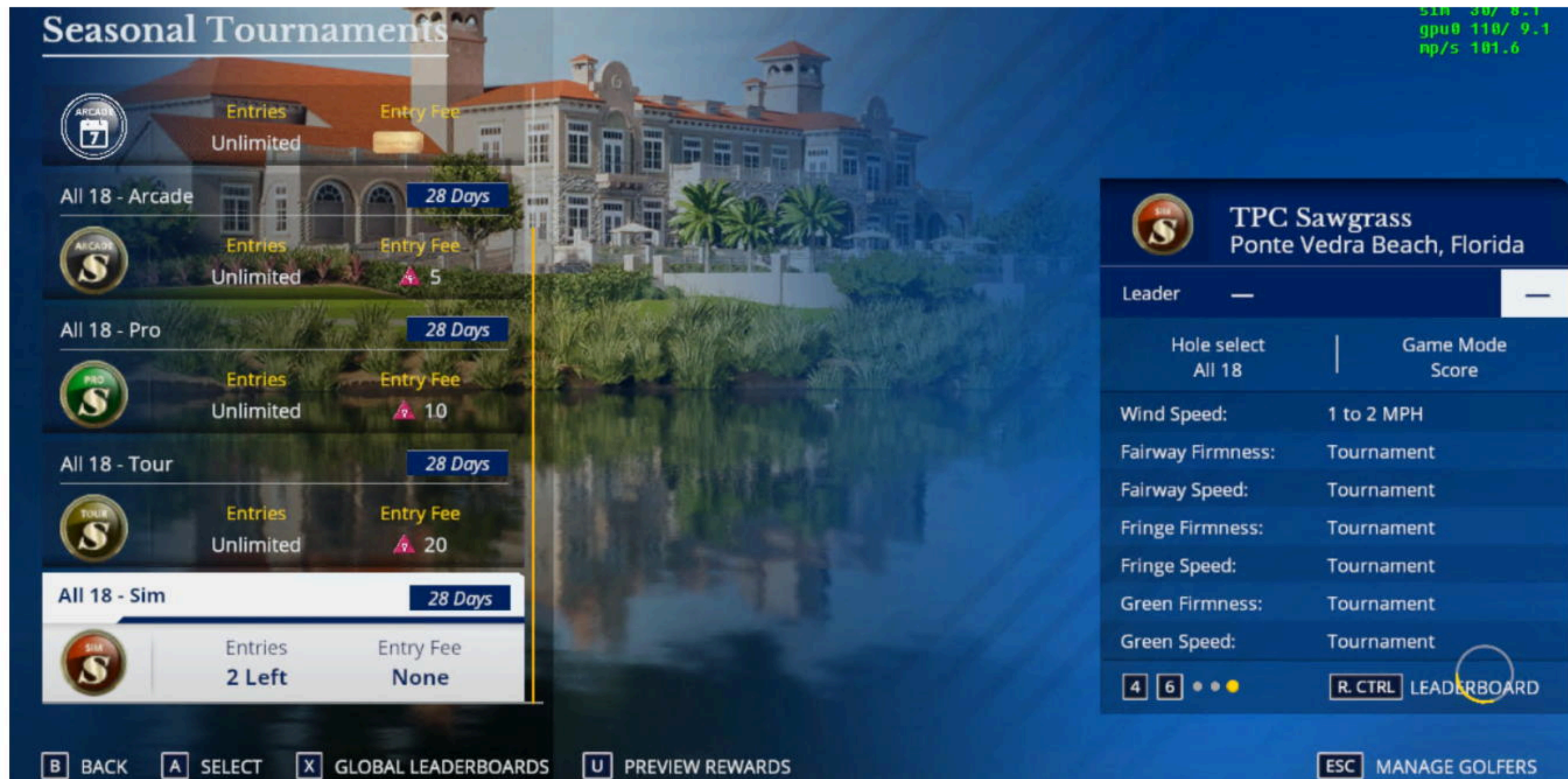
Quests and Challenges

I have been in charge of delivering a full system to design and implement different types of quests and challenges in the game. This system has helped content designers to design and implement quests and challenges following a pipeline based on 3 main steps: design, implementation and testing. All this process flows through a database driven by SQL, Excel and custom tools.



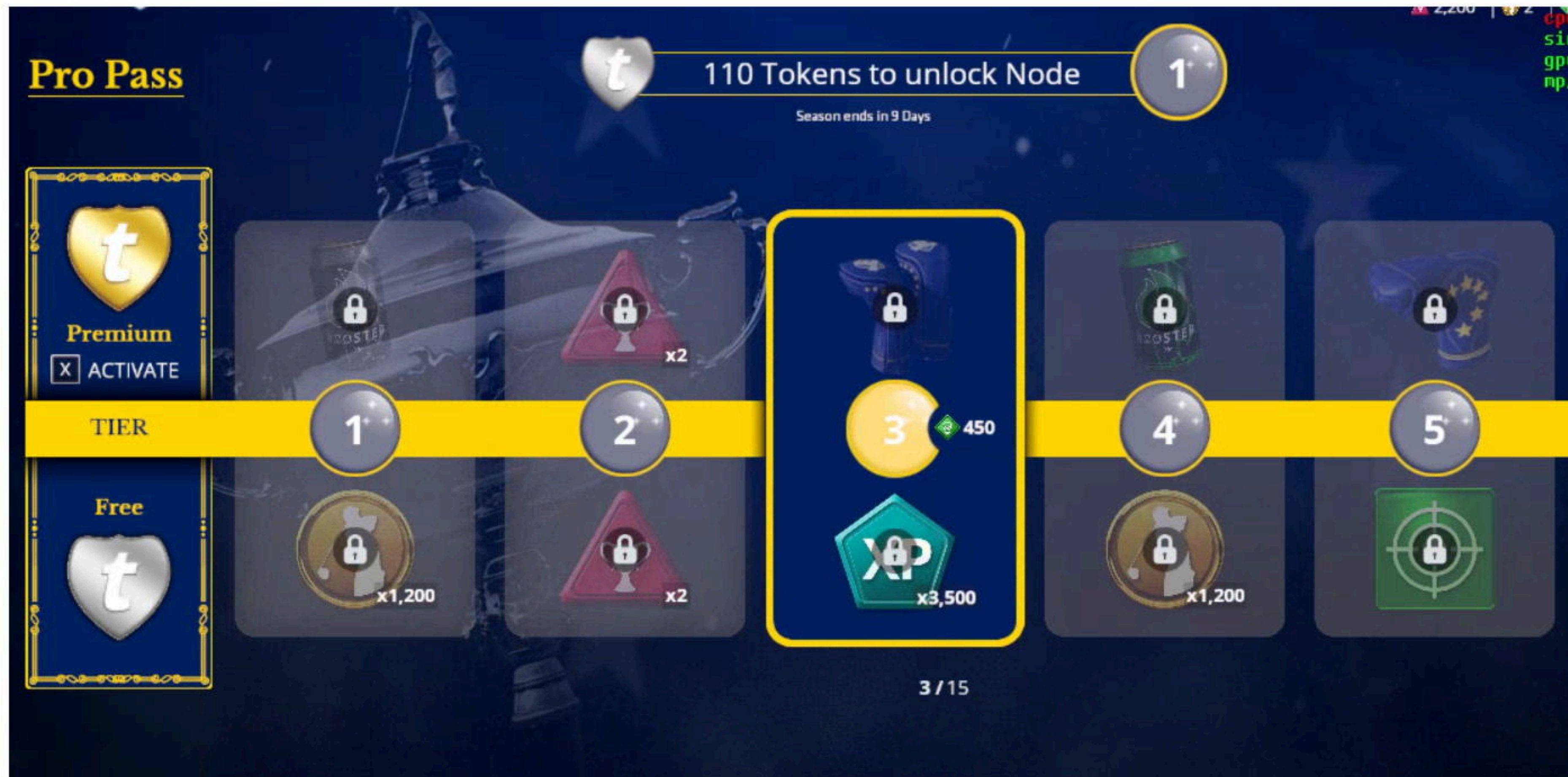
Tournaments

Designing a full pipeline to implement the Tournaments for each season. Content designers have been able to implement different types of tournaments: Daily, weekly and seasonal.



Event Pass

In order to increase the monetization in live services and empower the rest of game systems, we have designed and implemented a full Event Pass system to centralize all the rewards from the game in a specific place. This system let us to implement different event passes based on the current season.



Player Progression and Skill Tree System

I was in charge of designing a full balancing system based on power progression curves for the XP and Leveling of the player. At the same time, I was in charge of the balancing of the skills and attributes of the player when leveling up the character.



Progression, Skills and Ball-Striking Systems

Across PGA TOUR, my work connected numerical progression with the player-facing experience: level growth, attribute investment, skill unlocks and the systems that shape how the golfer develops over time.

PLAYER PROGRESSION

XP and level progression built from balancing curves and formulas.

SKILLS & ABILITIES

Skill Point investment, attributes and ability unlocks connected to character growth.

BALL STRIKING

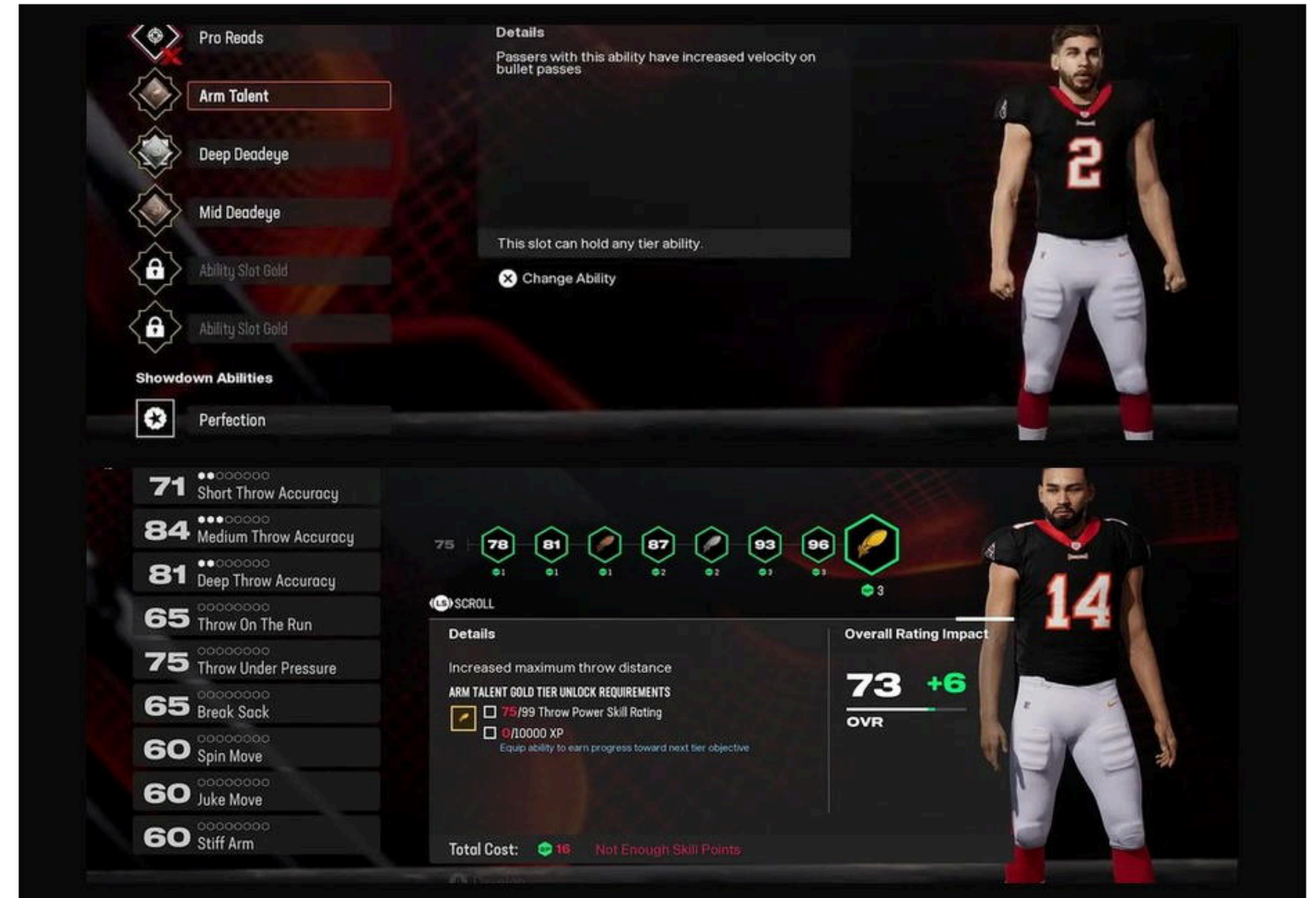
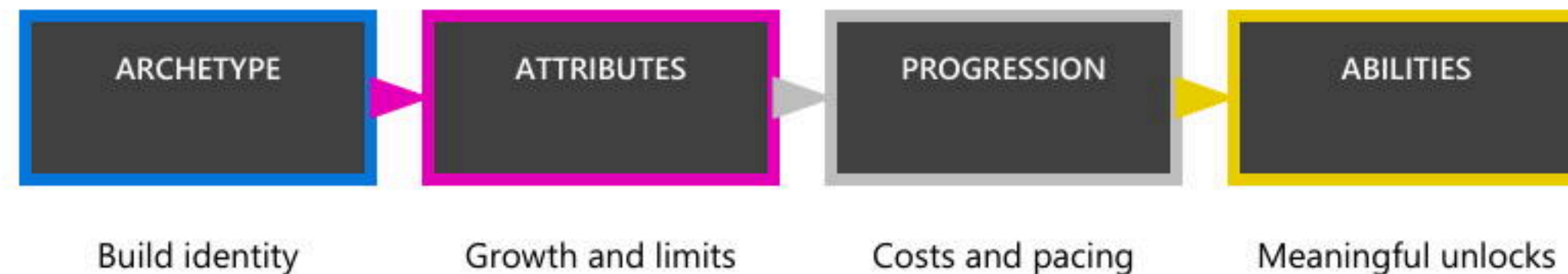
Gameplay and UI work supporting the player-facing ball-striking experience.



Archetypes, Progression & Player Abilities

For Madden NFL 25, I worked on Superstar archetypes, player progression and abilities through release. The design challenge was to make each build feel distinct while keeping growth, costs and unlock pacing coherent across the career experience.

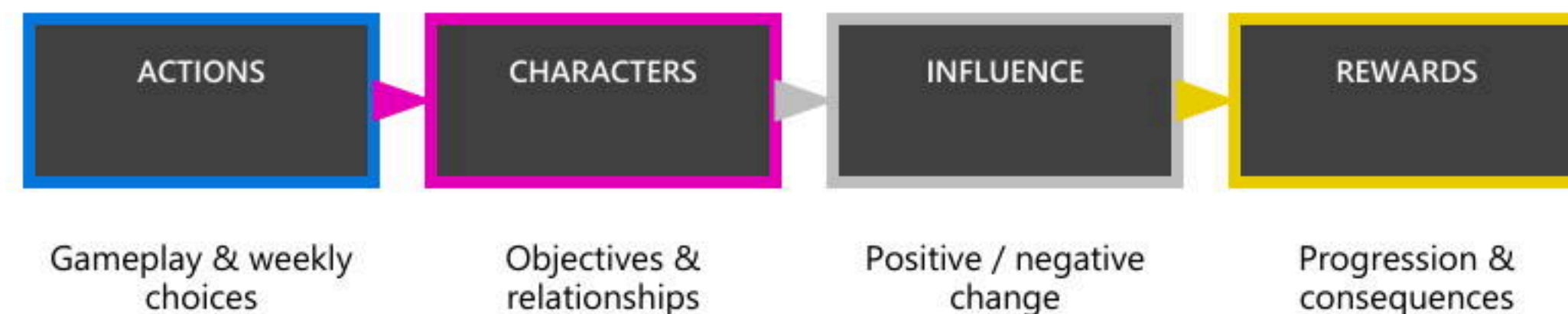
My work connected archetype identity with attribute progression and ability access, supporting balancing and iteration as the system moved toward the shipped game.



Sphere of Influence

For Madden NFL 26, I worked on Sphere of Influence, the RPG-inspired relationship and progression layer inside Superstar. The system connects what the player does on and off the field with the characters around them, creating progression through influence, relationship levels and rewards.

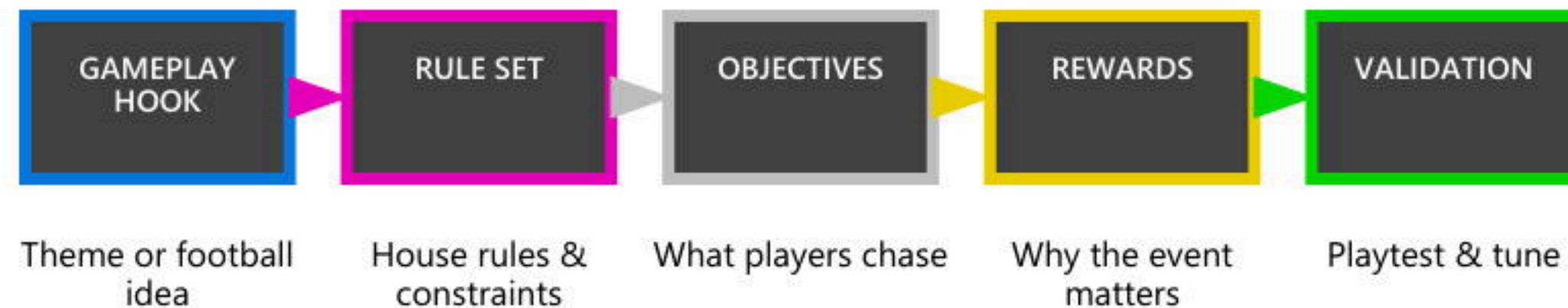
My focus was on making those relationships legible as a system: choices and gameplay actions feed character progression, which in turn changes the rewards and opportunities available to the player.



Live Services - Showdown Events

Within Madden NFL 26 Live Services, I designed and authored every Showdown event released across the year. Each event needed a clear gameplay hook that could be translated into rules, objectives, rewards and pacing while still feeling competitive and readable to players.

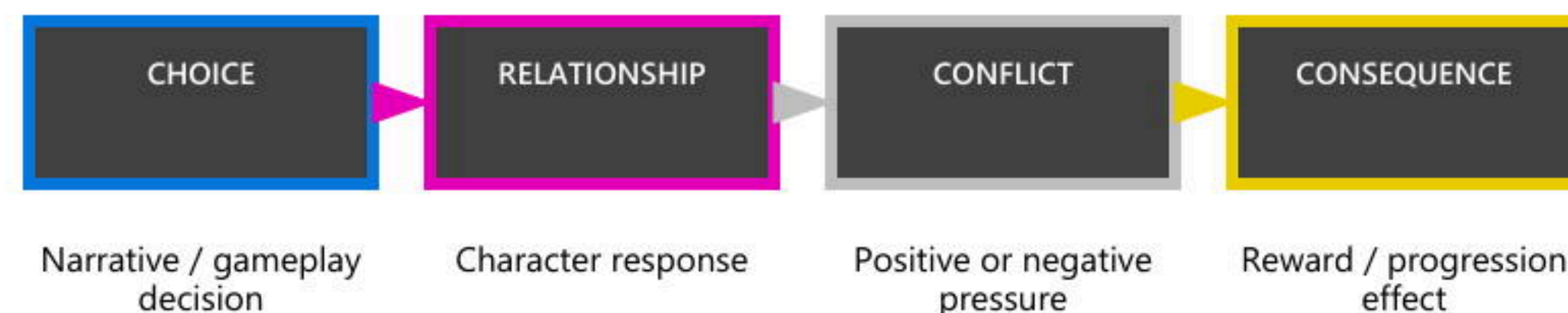
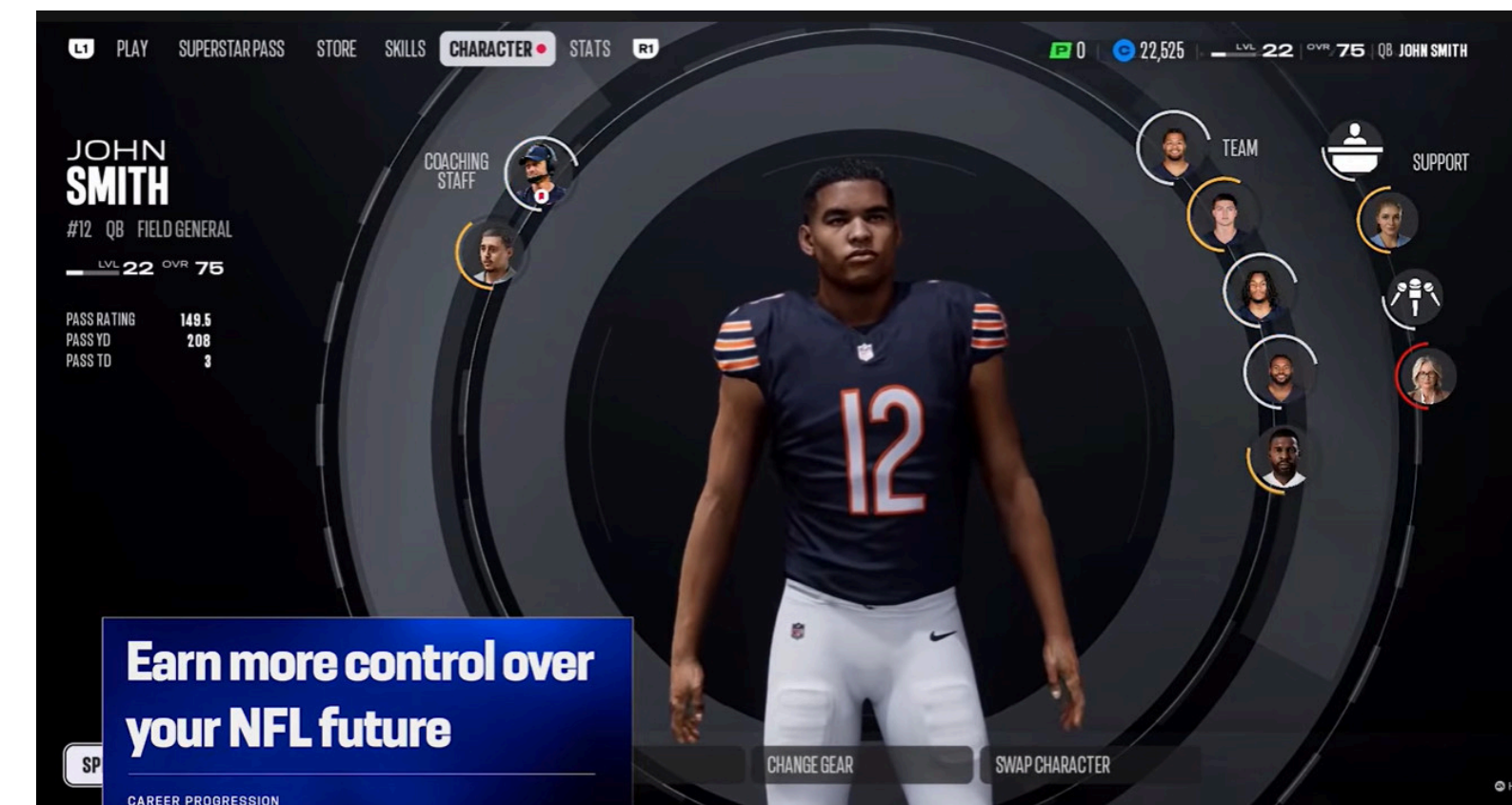
Beyond delivering the yearly event schedule, I also created new event proposals and gameplay variations to expand the existing Showdown formula and keep the mode fresh throughout the live-service cycle.



Sphere of Influence - System Evolution

For Madden NFL 27, I continued working on the evolution of Sphere of Influence before leaving EA SPORTS. The design expanded the relationship layer around player choices, character reactions, conflicts, rewards and longer-term career progression.

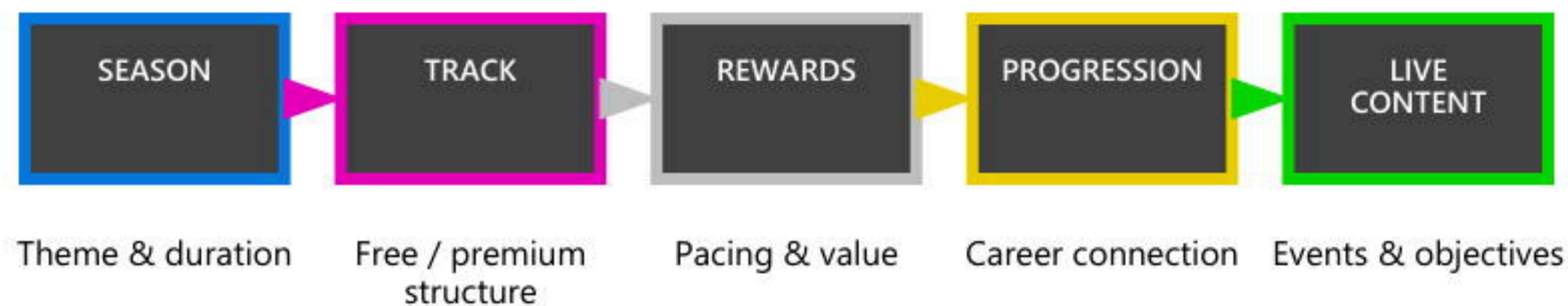
The goal was to strengthen the connection between narrative decisions and systemic consequences, so relationship changes were not isolated story beats but part of the player's ongoing Superstar build and career.



Battle Pass & Seasonal Progression

I also worked on the Battle Pass structure for Madden NFL 27 Superstar, defining how seasonal rewards and progression could support both short-term goals and the longer career loop.

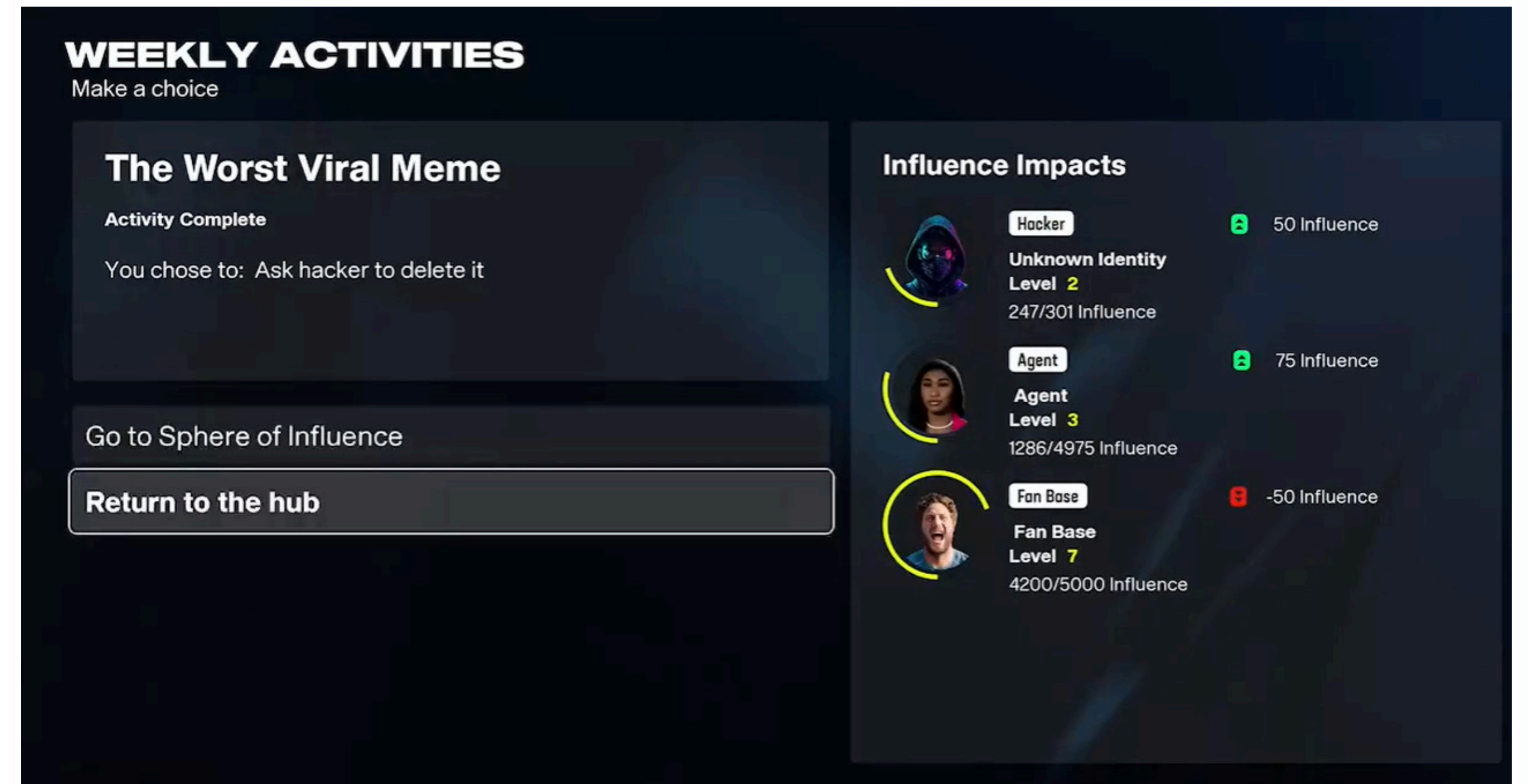
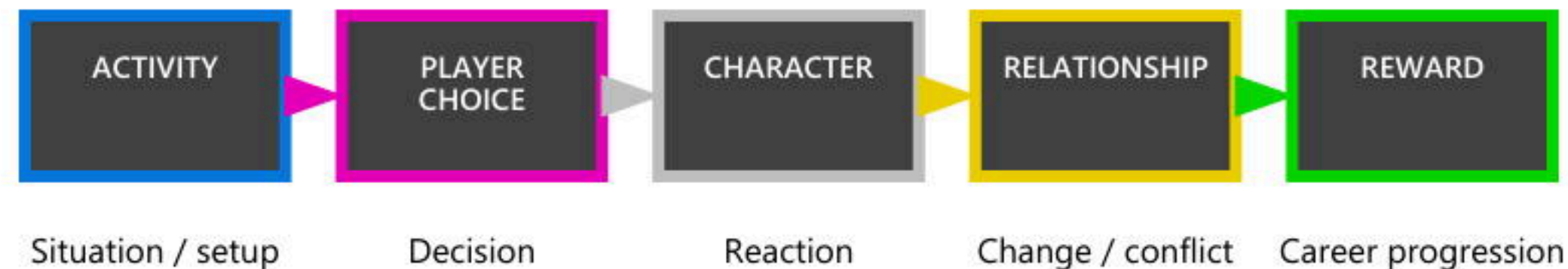
The system work focused on reward pacing, track structure and the relationship between live-service content and the broader progression economy, so each season could feel distinct without becoming disconnected from Superstar progression.



Side Activities - Narrative Systems

Side Activities were another major part of my Madden NFL 27 work. I designed off-field activities around choices, relationships, conflicts, rewards and progression, using them as smaller systemic situations that could feed back into the Superstar career.

The design challenge was to make each activity readable on its own while still contributing to the larger relationship and progression network rather than behaving as isolated narrative content.



EA SPORTS was the point where my earlier experience in implementation, balancing and simulation became part of large-scale AAA production. I learned to design systems not as isolated features, but as connected pieces of a product with dependencies across gameplay, progression, UI, data, production and live service.

Across PGA TOUR and three Madden production cycles, my role increasingly centred on system ownership, content architecture and iteration: translating design intent into structures that other disciplines could build, validate and sustain over time.

EA Sports
CONCLUSIONS